



DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING

MIGUEL CHEN

BSc in Electrical and Computer Engineering

ADVERSARIAL ATTACKS ON DRL BASED NETWORK ROUTING SYSTEMS

MASTER'S THESIS

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING
SPECIALIZATION IN TELECOMMUNICATIONS

NOVA University Lisbon
January, 2026



DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING

ADVERSARIAL ATTACKS ON DRL BASED NETWORK ROUTING SYSTEMS

MASTER'S THESIS

MIGUEL CHEN

BSc in Electrical and Computer Engineering

Supervisor: Pedro Miguel Figueiredo Amaral
Full Professor, NOVA University Lisbon

Examination Committee

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING
SPECIALIZATION IN TELECOMMUNICATIONS

NOVA University Lisbon
January, 2026

Adversarial Attacks On DRL Based Network Routing Systems

Master's Thesis

Copyright © Miguel Chen, NOVA School of School and Technology, NOVA University Lisbon.

The NOVA School of School and Technology and the NOVA University Lisbon have the right, perpetual and without geographical limitations, to archive and publish this dissertation in print or digital form, or by any other existing or future means, and to disseminate it through scientific repositories and to allow its copying and distribution for non-commercial educational or research purposes, provided that proper credit is given to the author and the publisher.

This document was produced with the pdfL^AT_EX [1] typesetting system and the NOVAthesis template (v7.9.1) [2].

- [1] The L^AT_EX Project team. *L^AT_EX - A document preparation system*. 1985. URL: <https://www.latex-project.org> (visited on 2025-12-29).
- [2] J. M. Lourenço. *The NOVAthesis L^AT_EX Template User's Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomlourengo/novathesis/raw/main/template.pdf>.

DISCLOSURE OF DELEGATION TO GENERATIVE ARTIFICIAL INTELLIGENCE

Miguel Chen declares¹ the use of Generative AI in the research work and writing process of this document. According to the GAIDeT taxonomy [3], the following tasks were delegated to Generative AI tools under full human supervision:

Conceptualization

- | | | |
|--|--|---|
| ☐ Idea generation | ☐ Formulating research questions and hypotheses | ☐ Feasibility assessment and risk evaluation |
| ☐ Defining the research objective | | ☐ Preliminary hypothesis testing |

Literature Review

- | | | |
|---|--|---|
| ☒ Literature search and systematization | ☒ Writing the literature review | ☐ Evaluation of novelty and identification of gaps |
| | ☐ Analysis of market trends and/or patent environment | |

Methodology

- | | | |
|--|---|--|
| ☐ Research design | ☒ Development of experimental or research protocols | ☐ Selection of research methods |
|--|---|--|

Software Development and Automation

- | | | |
|---|---|--|
| ☐ Code generation | ☐ Process automation | ☒ Creation of algorithms for data analysis |
| ☒ Code optimization | | |

Data Management

- | | | |
|--|---|--|
| ☐ Data collection | ☐ Data curation and organization | ☐ Reproducibility testing |
| ☐ Validation | ☐ Data analysis | |
| ☐ Data cleaning | ☐ Visualization | |

¹This declaration was generated using pdfL^AT_EX [1] and the L^AT_EX package aidisclose [2].

Writing and Editing

- | | | |
|--|--|--|
| ☒ Text generation | ☐ Formulation of conclusions | ☐ Translation |
| ☒ Proofreading and editing | ☐ Adapting and adjusting emotional tone | ☐ Reformatting |
| ☒ Summarizing text | | ☐ Press releases and outreach materials |

Ethics Review

- | | | |
|--|---|--|
| ☐ Bias analysis and discrimination assessment | ☐ Ethical risk analysis | ☐ Data confidentiality monitoring |
| | ☐ Monitoring compliance with ethical standards | |

Supervision

- | | | |
|---|--|--|
| ☐ Quality assessment | ☐ Identification of limitations | ☐ Publication support |
| ☐ Trend identification | ☐ Recommendations | |

Generative AI tools used: Claude Opus 5.

CONTENTS

Disclosure of Delegation to Generative Artificial Intelligence	iii
Contents	v
List of Figures	ix
List of Tables	xi
List of Algorithms	xiii
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	2
1.3 Approach	2
1.4 Contributions	3
1.5 Document Structure	4
2 Literature Review	5
2.1 Deep Reinforcement Learning	5
2.1.1 Deep Q-Networks	6
2.1.2 Duelling Q-networks	6
2.2 Multi-Agent Deep Deterministic Policy Gradient	7
2.2.1 MADDPG in network routing	7
2.3 Graph Neural Networks	8
2.3.1 GNN applications in network routing	8
2.4 Adversarial Attacks on Deep Reinforcement Learning	9
2.4.1 Gradient-based observation attacks: FGSM and PGD	9
2.4.2 Learned adversaries and the State-Adversarial MDP	11
2.4.3 Strategically-timed attacks	12
2.4.4 Multi-agent and joint attacks	12
2.4.5 Other attack surfaces	13
2.4.6 Summary of attacks	14
2.5 Adversarial Attacks on Graph Neural Networks	14

2.6	Summary and Research Gap	15
3	Methodology	17
3.1	Research Approach and Design	17
3.1.1	Research approach	17
3.1.2	Experimental design	18
3.2	Starting Point and Inherited Components	18
3.2.1	The routing system under test	19
3.2.2	The FGSM baseline	20
3.2.3	The learned-adversary scaffold	20
3.2.4	Contribution of this thesis	20
3.3	Victim Architectures Under Test	21
3.3.1	Central (CC) versus local (LC) critic	21
3.3.2	Simple versus duelling value head	22
3.3.3	With and without GNN encoding	22
3.3.4	Variant matrix	23
3.4	Coordinated and Strategically-Timed Attacks on MADDPG Routing Agents	23
3.4.1	Threat model and admissible perturbation set	23
3.4.2	(A) Coordinated multi-agent perturbation	25
3.4.3	(B) Strategically-timed attacks under an L_0 budget	27
3.4.4	Learned observation-space adversary	29
3.5	Data Collection	33
3.5.1	Environment and traffic regime	33
3.5.2	Training the adversaries	33
3.5.3	Evaluation runs	33
3.5.4	Recorded data	34
3.5.5	Implementation and platform	34
3.6	Data Analysis	35
3.6.1	Measures	35
3.6.2	Adversarial gap and confidence intervals	35
3.6.3	Derived comparisons	36
3.6.4	Damage per attacked step	37
3.6.5	Multiple comparisons	37
3.7	Validity and Reliability	37
3.7.1	Internal validity	37
3.7.2	Construct validity	38
3.7.3	Reliability	38
3.8	Limitations	38
3.9	Summary	39
4	Statistics and Results	41

4.1	Reported Statistic and Experimental Protocol	42
4.2	Results Under the FGSM-Parity Domain Clamp	43
4.2.1	Independent scope: sensitivity to the timing budget	43
4.2.2	Independent scope: decision flips versus delivered damage	45
4.2.3	Independent scope: what triggered the attacks	45
4.2.4	Coordinated scope	46
4.2.5	Effect of coordination	47
4.3	Comparison with the FGSM Baseline	48
4.3.1	Independent scope with timing budget	49
4.3.2	Coordinated scope	50
4.4	Results Under the Full Domain Clamp	51
4.4.1	Independent scope: sensitivity to the timing budget	51
4.4.2	Independent scope: decision flips versus delivered damage	53
4.4.3	Coordinated scope	54
4.4.4	Effect of coordination	55
4.5	Effect of the Domain Clamp	57
5	Conclusions and Future Work	59
5.1	Conclusions	59
5.1.1	Effectiveness compared with the FGSM baseline	59
5.1.2	Coordination compared with the independent attack	60
5.1.3	Timing budget compared with attacking every step	60
5.1.4	Further observations	61
5.2	Limitations	61
5.2.1	Statistical relevance of the results	61
5.2.2	Scope of the evaluation	62
5.2.3	Design of the attack and its control	62
5.3	Future Work	62
5.3.1	Repeated training for statistical relevance	62
5.3.2	Optimisation-based PGD and SAJA attacks	63
	Bibliography	65
	Appendices	
A	Timing Gate Triggers	73

LIST OF FIGURES

2.1	Adversarial attacks on MADDPG and related multi-agent learners, grouped by the part of the system they perturb.	9
4.1	Independent scope, FGSM-parity domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes. Left: gap against the requested timing budget, with the mean realised attack rate printed beneath each tick and the ungated run included as the 100% reference; filled markers denote a 95% confidence interval excluding zero, hollow markers a null. Right: the same gaps plotted against the fraction of routing decisions the attack actually changed, on a logarithmic axis, with each variant drawn as one path walking from $b = 0.10$ through to the ungated run. A thin line joins a label to its series where the label had to be moved clear of the data.	43
4.2	Coordinated scope, FGSM-parity domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes, with the perturbation emitted jointly for the whole compromised group. Panels and markers as in Figure 4.1. A thin line joins a label to its series where the label had to be moved clear of the data.	46
4.3	Independent scope, full domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes. Left: gap against the requested timing budget, with the mean realised attack rate printed beneath each tick and the ungated run included as the 100% reference; filled markers denote a 95% confidence interval excluding zero, hollow markers a null. Right: the same gaps plotted against the fraction of routing decisions the attack actually changed, on a logarithmic axis, with each variant drawn as one path walking from $b = 0.10$ through to the ungated run.	52
4.4	Coordinated scope, full domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes, with the perturbation emitted jointly for the whole compromised group. Panels and markers as in Figure 4.3. A thin line joins a label to its series where the label had to be moved clear of the data.	54

A.1	Independent scope, FGSM-parity domain clamp: why the timing gate fired in the attack arm ($\varepsilon = 0.30$, $n = 30$ episodes). One panel per trigger that occurred in at least one run, combining the case of Equation (3.10) that admitted the step with the congestion band of its score. Rows are victim variants and columns the requested timing budget. Each cell gives the trigger's share of that run's attacked steps, with the count beneath it, so the three panels of one cell sum to 100%. Ungated runs have no gate and are omitted.	73
A.2	Coordinated scope, FGSM-parity domain clamp: why the timing gate fired in the attack arm ($\varepsilon = 0.30$, $n = 30$ episodes). Panels, rows, columns and cell contents as in Figure A.1.	74

LIST OF TABLES

2.1	Adversarial attacks on MADDPG and related learners: observation, action and communication space.	14
2.2	Adversarial attacks on MADDPG and related learners: training-time and transfer attacks.	15
3.1	Research questions and the comparisons that answer them. The measures are defined in Section 3.6.	19
3.2	MADDPG victim variants under test	23
3.3	What each attack needs to know or access. The learned adversary’s requirements differ between training and attack time; the timing gate’s requirement applies only when a timing budget is used.	25
4.1	Independent scope, FGSM-parity domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero. Positive values indicate the attack cost the victim more than the random control drawn from the same ε -ball; negative values indicate the reverse.	44
4.2	Independent scope, FGSM-parity domain clamp. Ungated results at $\varepsilon = 0.30$, $n = 30$ paired episodes: adversarial gap with its 95% confidence interval, and the fraction of routing decisions changed by the attack and by the random control.	45
4.3	Coordinated scope, FGSM-parity domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero.	47
4.4	Effect of coordination under the FGSM-parity domain clamp: coordinated minus independent adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean coordination increased the damage. Bold entries have a 95% confidence interval on the paired difference excluding zero.	48

4.5	FGSM baseline under the FGSM-parity domain clamp, $\varepsilon = 0.30$, no link failures, $n = 15$ paired episodes: adversarial gap against FGSM's own random control, with its 95% confidence interval, and the fraction of routing decisions the attack changed. Bold entries have a confidence interval excluding zero.	49
4.6	Independent learned adversary against FGSM, FGSM-parity domain clamp: Δ^{FGSM} of Equation (3.19), in percentage points, by victim variant and the learned adversary's requested timing budget ($\varepsilon = 0.30$, $n = 15$ paired episodes). FGSM attacks every step in every column. Positive values mean the learned adversary did more damage. Bold entries have a 95% confidence interval excluding zero.	49
4.7	Coordinated learned adversary against FGSM, FGSM-parity domain clamp: Δ^{FGSM} of Equation (3.19), in percentage points, by victim variant and the learned adversary's requested timing budget ($\varepsilon = 0.30$, $n = 15$ paired episodes). Positive values mean the learned adversary did more damage. Bold entries have a 95% confidence interval excluding zero.	50
4.8	Independent scope, full domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero. Positive values indicate the attack cost the victim more than the random control drawn from the same ε -ball; negative values indicate the reverse.	52
4.9	Independent scope, full domain clamp. Ungated results at $\varepsilon = 0.30$, $n = 30$ paired episodes: adversarial gap with its 95% confidence interval, and the fraction of routing decisions changed by the attack and by the random control.	53
4.10	Coordinated scope, full domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero.	55
4.11	Effect of coordination under the full domain clamp: coordinated minus independent adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean coordination increased the damage. Bold entries have a 95% confidence interval on the paired difference of Equation (3.17) excluding zero.	56
4.12	Effect of the domain clamp in the independent scope: FGSM-parity minus full clamp adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean the parity-clamp adversary did more damage. Bold entries have a 95% confidence interval on the paired difference of Equation (3.18) excluding zero.	57
4.13	Effect of the domain clamp in the coordinated scope: FGSM-parity minus full clamp adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean the parity-clamp adversary did more damage. Bold entries have a 95% confidence interval on the paired difference of Equation (3.18) excluding zero.	58

LIST OF ALGORITHMS

1	Critical-state timing gate with rationed ties	29
2	Training the learned observation-space adversary (independent scope) . .	31

INTRODUCTION

1.1 Context and Motivation

As telecommunication networks grow in scale and heterogeneity, routing, quality of service and resource management become increasingly difficult to handle with hand-crafted rules. Traditional routing protocols such as OSPF and BGP rely on static, rule-based decisions that adapt poorly to variable traffic, heterogeneous Quality-of-Service (QoS) requirements and competing optimisation constraints. Deep Reinforcement Learning (DRL) has emerged as an alternative, allowing routing agents to learn adaptive policies directly from observations of the network rather than from explicitly programmed rules [4].

Multi-agent formulations extend this to distributed control, in which every forwarding node is an agent that decides locally while the paths chosen collectively determine end-to-end delivery. Multi-Agent Deep Deterministic Policy Gradient (MADDPG) is a natural fit for this setting: under centralised training with decentralised execution, each agent learns from a critic that may observe the whole system during training, yet acts on its own local observation once deployed [5]. Combined with Graph Neural Networks, which exploit the graph structure of the topology, such controllers have been reported to improve scalability, topology awareness and generalisation across network configurations [6–11].

The same property that makes these controllers adaptive also exposes them. A learned routing policy decides from the network state it perceives, and neural policies are known to be sensitive to small, deliberately chosen perturbations of their inputs [12, 13]. Attacks such as the Fast Gradient Sign Method (FGSM) [13], Projected Gradient Descent (PGD) [14] and the Carlini and Wagner attack [15] were developed largely for image classification, where one perturbation targets one prediction. In reinforcement learning the stakes differ: a perturbed observation changes an action, the action changes the states that follow, and the damage accumulates over a trajectory [16]. For a routing controller, an adversary who can corrupt the telemetry an agent receives, without ever touching the network itself, can redirect traffic. How vulnerable learned routing controllers are to such an adversary remains largely unexplored.

1.2 Problem Statement

This thesis continues a line of research on a MADDPG routing controller developed in earlier work [17]. A preceding dissertation in the same line attacked that controller with FGSM, perturbing each agent’s observation in a single gradient step at every time step [TODO-colleague-fgsm]. The attack altered a substantial fraction of the routing decisions taken by the victim, yet recovered only a small part of the degradation achievable in principle, because the redundancy of the topology absorbed the redirected traffic.

That result admits two readings. Either the controller is genuinely robust, or a single-step, per-agent adversary is too weak for its failure to count as evidence of robustness. The two cannot be separated without an attacker that is free of those two limitations and theoretically stronger. This thesis builds one and compares it with FGSM on identical episodes and within the same admissible perturbation set. The central research question is therefore:

Does the apparent robustness of MADDPG routing controllers to FGSM survive a theoretically stronger observation-space attack, one that coordinates its perturbation across agents and divides it across time?

It is broken down into four questions:

- RQ1** Does dividing the attack across time, spending a limited budget on the network’s most critical states, buy more damage per attacked step than perturbing every step?
- RQ2** Does coordinating the perturbation across compromised agents, rather than attacking each agent in isolation, increase the damage?
- RQ3** Which architectural choices of the victim, namely its critic domain, its value head and the use of GNN encoding, determine how vulnerable it is?
- RQ4** Is the fraction of routing decisions an attack changes a meaningful measure of the damage it does?

1.3 Approach

The attack addresses the two weaknesses of the baseline in turn, drawing on a different source of inspiration for each. Where FGSM spends its whole perturbation budget in a single step, PGD divides it into a sequence of smaller projected steps [14]. The attack carries this principle from the input into time: instead of applying a full-strength perturbation at every step, it acts on only a fraction of the steps, chosen by a critical-state score that identifies the moments at which the network’s redundancy is thinnest. Where the baseline perturbs each agent in isolation, SAJA [18] judges the perturbations of several victim agents together, by the centralised critic’s value of the joint action they induce. Inspired by it, the

attack coordinates its perturbation jointly across the compromised agents, so that several of them can be steered towards the same shared link.

The attack is realised as the learned observation adversary of the State-Adversarial MDP [16]. Because the victim is frozen, its policies are fixed, and against a fixed policy the problem of finding the optimal observation adversary is itself an ordinary Markov decision process. The adversary is therefore trained by reinforcement learning to act in this coordinated and strategically timed way, learning which perturbation to apply from the damage its perturbations cause. In principle this makes it stronger than FGSM, since within the same admissible set the optimal such adversary is at least as damaging, by its own measure of damage, as any deterministic per-step perturbation rule, FGSM included. It also knows less about the victim: it never needs the victim’s gradients, only its actions during training. It does, however, need a faithful simulator of the network to train against and, when its attack is timed, the utilisation of every link while it attacks.

The attack is evaluated against seven victim variants that differ in their critic domain, their value head and the use of GNN encoding. To keep the comparison with the FGSM study meaningful, the attack operates on the same frozen agents and within the same ℓ_∞ perturbation budget, and is scored under the same evaluation protocol. It is evaluated under the FGSM study’s domain constraint and, as a second condition, under a stricter constraint that keeps every observation feature within $[0, 1]$, with a separately trained adversary for each. Every configuration is run as three paired arms on identical traffic: a clean arm, a random control that draws a random perturbation from the same admissible set, and the attack. The damage attributed to the attack is its margin over the random control, since any perturbation within that set disturbs the controller whether or not its direction was chosen adversarially.

1.4 Contributions

The main contributions of this thesis are:

- An observation-space attack on MADDPG routing controllers that extends the learned adversary of the State-Adversarial MDP [16] with coordination of the perturbation across agents, inspired by SAJA, and division of the attack across time, inspired by PGD, together with an explicit threat model of what the attacker must know and control.
- A critical-state timing gate that spends a limited attack budget on high-leverage moments. It includes a tie-rationing rule that makes the realised attack rate match the requested budget, which a plain threshold cannot do because the critical-state score concentrates on a few repeated values.
- An evaluation of the learned adversary against a paired random control that shares its admissible set and timing gate, which separates the damage attributable to the

attacker’s choice of perturbation from that of a random perturbation within the same set, and which reports decisions changed separately from delivery lost.

- An empirical study across seven victim architectures, both attack scopes and two domain constraints, which finds that the architectural choice that appears vulnerable depends on the constraint and the scope.
- A paired comparison with the FGSM baseline on identical episodes and within the FGSM study’s admissible set, measured against its own random control.

1.5 Document Structure

The remainder of this document is organised as follows. Chapter 2 reviews the relevant literature: deep reinforcement learning and its application to routing, MADDPG, graph neural networks and their use in routing, and adversarial attacks on reinforcement learning agents, from FGSM and PGD to learned State-Adversarial MDP adversaries, strategically-timed attacks and SAJA, and it closes by identifying the gap this thesis addresses. Chapter 3 sets out the research approach and experimental design, the routing system and FGSM baseline inherited from earlier work, the victim architectures under test, the attack with its SAJA-inspired coordination and PGD-inspired timing mechanisms and the learned adversary that realises it, and then how the data were collected and analysed, how the validity of the results was protected, and the delimitations of the study. Chapter 4 presents the results under the FGSM study’s domain constraint and under the stricter one, reporting the independent and coordinated scopes, the comparison with the FGSM baseline, the effect of coordination and the effect of the constraint itself. Chapter 5 concludes, discusses the limitations of the results and outlines directions for future work. Appendix A gives the heatmaps of why the timing gate fired.

LITERATURE REVIEW

This chapter reviews the literature on which this thesis builds, organised around its two subjects: learning-based routing controllers, which are the victims, and adversarial attacks on reinforcement learning agents, which are the tools used against them. Works were selected for their relevance to the system under test, a multi-agent routing controller trained with MADDPG and optionally encoded with a graph neural network, and to the attack developed here, a learned observation-space adversary with a coordination mechanism and a timing mechanism. Section 2.1 introduces deep reinforcement learning and its value-based methods, Section 2.2 the multi-agent algorithm used by the victim and its applications to routing, and Section 2.3 graph neural networks and their use in routing. Section 2.4 then reviews attacks on reinforcement learning agents, from single-step gradient attacks to learned, timed and multi-agent attacks, and Section 2.5 briefly situates attacks on graph neural networks. Section 2.6 draws these strands together and identifies the gap this thesis addresses.

2.1 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines reinforcement learning with deep neural networks. An agent interacts with an environment, choosing actions from its observations and receiving rewards, and learns a policy that maximises its expected cumulative reward. Because the policy is learned from observations rather than programmed, DRL suits network optimisation problems whose dynamics are too complex for hand-crafted rules [19–21]. In routing, DRL agents learn forwarding policies that adapt to congestion, link failures and changing demand without an explicit model of the network [22–24]. The problem is usually modelled as a Markov decision process, or a partially observable one, in which states describe link loads and topology, actions select next hops or paths, and rewards reflect delay, throughput or a composite quality-of-service measure [25].

2.1.1 Deep Q-Networks

Deep Q-Networks (DQN) combine Q-learning with a deep neural network that approximates the action-value function $Q(s, a; \theta)$, taking an observation as input and producing an estimate of the expected discounted return of every discrete action as output [26, 27]. The agent acts greedily with respect to these estimates, with occasional random actions for exploration, and the network is trained to minimise the temporal-difference error

$$L(\theta) = \mathbb{E}_{(s,a,r,s')} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right], \quad (2.1)$$

where γ is the discount factor and θ^- are the parameters of a target network that is updated only periodically. Two mechanisms make this training stable: experience replay, which draws the transitions (s, a, r, s') at random from a memory buffer to break the correlation between consecutive samples, and the slowly updated target network, which keeps the regression target from shifting at every update [27].

Its main advantage is that it learns control policies directly from high-dimensional observations, without hand-crafted features, and reuses past experience through replay; this has made it a common basis for learned routing, for example in underwater sensor networks [28] and in software-defined data centres [29]. Its main limitations follow from the same design. Because the network outputs one value per action and the target maximises over them, DQN applies only to discrete action spaces of moderate size. Training still needs a large amount of interaction with the environment, and the replay buffer and target network are needed precisely because combining neural networks with temporal-difference learning is otherwise unstable.

2.1.2 Duelling Q-networks

The duelling architecture changes only the network of a DQN, not its training algorithm [30]. After a shared set of feature layers, the network splits into two streams: a value stream $V(s)$ that estimates how good the state is, and an advantage stream $A(s, a)$ that estimates how much better each action is than the others in that state. The two are recombined into action values as

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right), \quad (2.2)$$

where subtracting the mean advantage makes the split between V and A identifiable.

The advantage of this separation is that the value of a state can be learned from every transition, without having to learn the effect of each action separately. Learning is therefore more efficient in states where the choice of action matters little, and the benefit grows with the number of actions that have similar values [30, 31]. This property has been used in routing, for instance to allocate routes and spectrum in elastic optical networks, where many routing actions are of similar merit [32]. Its limitations are those of DQN, since it remains a value-based method for discrete actions, together with a larger network and a gain that is small when the actions' values differ sharply. The duelling idea also appears in

the victims of this thesis, whose critics use a value and an advantage stream (Section 3.3.2), although in a form that differs from the original.

2.2 Multi-Agent Deep Deterministic Policy Gradient

Multi-Agent Deep Deterministic Policy Gradient (MADDPG) extends actor-critic learning to settings with several agents through *centralised training with decentralised execution* (CTDE) [5]. Each agent has an actor that acts on its own local observation and a critic that, during training only, conditions on the actions of all agents. Conditioning on the joint action removes the non-stationarity that each agent would otherwise perceive as its peers' policies change, which is the main obstacle to training independent learners. Lowe et al. also propose two optional extensions: approximating the other agents' policies from their observed actions, for when those policies are not known, and training an ensemble of sub-policies per agent to make policies robust to changing opponents in competitive settings [5].

The same execution model defines the attack surface studied in this thesis. After training the critics are no longer used, and each actor maps its own observation to an action with no exchange of information with its peers. An adversary who corrupts the observations therefore acts on the policies directly, and a corrupted observation at one agent is not corrected by the others [25, 33, 34].

2.2.1 MADDPG in network routing

MADDPG has been applied to several network control problems. Tian et al. [4] treat each virtual network function of a 5G network slice as an agent that allocates resources and migrates as demand changes, and report better performance than approaches that do not use MADDPG. Huang et al. [8] cast routing in software-defined networks as a cooperative multi-agent problem in their ERA-MADDPG scheme: agents select paths from observed traffic and topology, the centralised critic uses the global state during training, and the decentralised actors adapt routes to changing traffic, which the authors find more efficient to train and faster to recover under topology changes than other DRL methods. RS-MADDPG [35] maps each node's local observation to continuous routing decisions, such as splitting ratios or next hops, and converges about 30 training cycles earlier than its baselines. DRAMA [10] keeps MADDPG's CTDE training but replaces the critic's fixed joint observation-action input with a Q-network that scores actions from topology-aware latent features and a graph-based emergent communication layer, so that the trained network can score newly appearing neighbours and cope with link failures without retraining; it reports higher delivery rates and lower latency than standard MADDPG across traffic loads and dynamic topologies.

Taken together, these works agree that CTDE suits routing: a centralised critic stabilises training, while decentralised actors make fast local decisions that scale with the network.

They differ in the problem addressed, from slice resource allocation to packet routing, in the action representation, discrete or continuous, and in the baselines and metrics against which they report gains, so their improvements are not directly comparable. They also share an assumption: every one of them evaluates its controller on the observations the network actually produces. None examines how the learned policies behave when those observations are corrupted, so the robustness of MADDPG routing controllers to adversarial inputs remains largely unexplored.

2.3 Graph Neural Networks

Graph Neural Networks (GNNs) are deep learning models that operate directly on graph-structured data. Each layer updates a node’s representation by aggregating, or passing messages from, the representations of its neighbours, so that after several layers every node’s representation encodes both its own attributes and the structure of its surrounding graph, with the same parameters shared across all nodes [36–39]. Surveys of the field distinguish several families, including recurrent GNNs, spectral and spatial convolutional GNNs, graph autoencoders and spatio-temporal GNNs, which trade off expressivity, scalability and ease of training [36, 38]. This thesis relies on none of the more specialised variants: the GNN-encoded victims of Section 3.3.3 use a standard spatial message-passing encoder of two graph convolution layers with a residual connection.

2.3.1 GNN applications in network routing

GNNs have been applied to routing and traffic engineering along three main lines [40]. The first uses GNNs as fast surrogate models of the network: Rusek et al. [41] show that message-passing GNNs can estimate per-link delay and loss accurately enough to evaluate many candidate routing schemes without full simulation, and later work feeds such predictions into external optimisers [40]. The second couples a GNN encoder with DRL to learn routing policies directly. G-Routing encodes the topology, traffic matrix and routing state and passes predicted path-level metrics to a DRL agent, converging faster and behaving more stably than shortest-path and traditional baselines under changing traffic and failures [42]; GROM learns traffic-splitting policies with a GNN and DRL, improving training stability and generalisation to unseen topologies [43]. The third designs GNN architectures around the structure of the traffic-engineering problem itself, for example by representing paths, demands, links and the global objective as nodes of one heterogeneous graph, which lets the model generalise to unseen topologies while approximately respecting constraints [44]. A broader survey [45] places these models within GNN-based control and optimisation, where routing and traffic engineering are among the most studied applications.

The recurring argument across these works is generalisation: because a GNN’s parameters are shared across nodes, a model trained on one topology can be applied to

another. The same property has a consequence for robustness that is not the focus of these works. Since each node’s representation is computed from its neighbours’, a perturbation of one node’s input propagates to the representations of the nodes around it, so a GNN encoder changes how an attack on one agent’s observations reaches the others.

2.4 Adversarial Attacks on Deep Reinforcement Learning

Neural networks are vulnerable to adversarial examples, inputs altered by small, deliberately chosen perturbations that change the model’s output [12]. DRL policies inherit this vulnerability, and attacks against them lead to poor decisions across many application areas [46–48]. Attacks introduced at test time are called evasion attacks, and those introduced during training poisoning attacks. Attacks on multi-agent systems such as MADDPG can further be classified by the part of the system they perturb (observations, actions, inter-agent communication, or the training process), by when they act, and by what the attacker knows about the victim, from full white-box access to its parameters to black-box query access only. Figure 2.1 organises the attacks reviewed below by the part of the system they perturb.

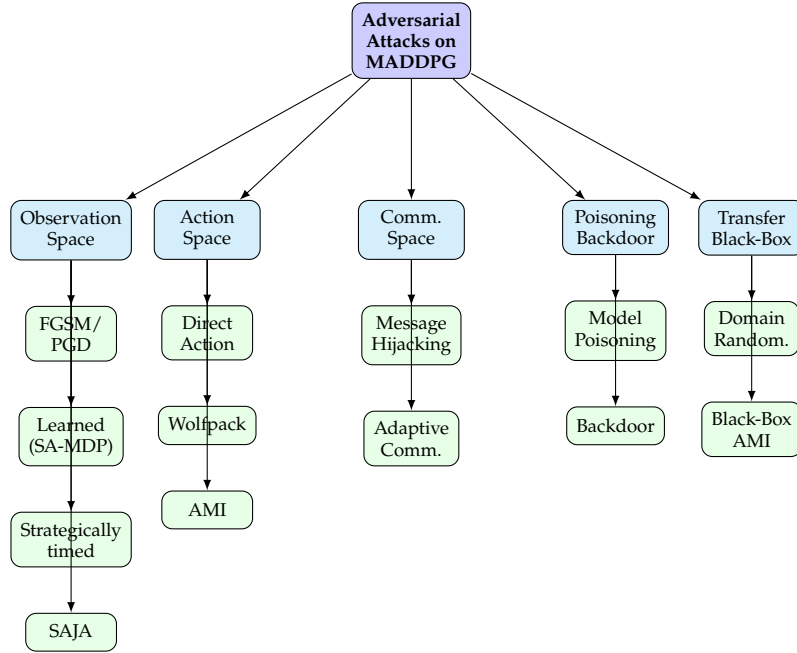


Figure 2.1: Adversarial attacks on MADDPG and related multi-agent learners, grouped by the part of the system they perturb.

2.4.1 Gradient-based observation attacks: FGSM and PGD

The **Fast Gradient Sign Method (FGSM)** perturbs an input x by a single step of size ϵ along the sign of the gradient of the model’s loss J ,

$$\tilde{x} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)), \quad (2.3)$$

where θ are the model’s parameters and y the input’s label [13]. Goodfellow et al. attribute its effectiveness to the largely linear behaviour of neural networks in high dimensions, contradicting the earlier view that adversarial vulnerability stems from their non-linearity, and FGSM has been reported not only to mislead models but to leave them confident in their mistakes [49]. Its cost of one gradient evaluation has made it the standard baseline from which many later attacks developed [50]. Kurakin et al. apply it iteratively as the Basic Iterative Method, taking several smaller steps and clipping the result to the ε -neighbourhood of the input after each, and add a targeted variant that moves towards the least likely class [51]; other extensions target specific classes through the model’s internal features [52] or improve transfer to unseen models by randomly transforming the input at each iteration [53].

Projected Gradient Descent (PGD), introduced by Madry et al. [14], generalises this into a multi-step attack. Starting from a random point in the allowed perturbation set S around x , it repeatedly takes a signed gradient step of size α and projects the result back onto $x + S$,

$$x_{t+1} = \Pi_{x+S}(x_t + \alpha \operatorname{sign}(\nabla_x L(\theta, x_t, y))). \quad (2.4)$$

Madry et al. study mainly the ℓ_∞ ball, which bounds the change to each input feature and under which a single step of size ε recovers FGSM, with additional experiments under the ℓ_2 ball. Across many random restarts PGD finds local maxima of very similar loss, which leads the authors to regard it as a universal first-order adversary: robustness to PGD indicates robustness to any attack that uses only gradient information [14].

The comparison between the two methods matters for how an attack’s failure should be read. Kurakin et al. found that models adversarially trained on single-step examples can perform better on those examples than on clean inputs, a “label leaking” effect they attribute to the examples being built from the true label [50, 54]. Madry et al. observe that networks trained against FGSM overfit to the narrow set of examples it produces and show no robustness against PGD, and conclude that robustness must be judged against a strong multi-step attacker rather than a single-step one [14]. A single-step attack that fails therefore says little about the model’s robustness. PGD is the inspiration for the timing mechanism of this thesis (Section 3.4.3): where FGSM spends its budget in one step and PGD divides it into several, the attack divides its effort across time, while every attacked step remains bounded by the full ε .

Both methods were, however, designed for classifiers, where one perturbation targets one prediction, and they carry three limitations into reinforcement learning. They are white-box, requiring the gradients of the victim. They choose each perturbation greedily at a single time step, although a perturbed action changes the states that follow. And applied to multi-agent learners they perturb each agent separately. Against MADDPG, Guo et al. report that FGSM is of limited effectiveness, because gradients derived from imperfectly trained critics give unreliable guidance [18, 55]. The works reviewed next each address one of these limitations.

2.4.2 Learned adversaries and the State-Adversarial MDP

In reinforcement learning, a perturbation that alters one action also alters the trajectory that follows, so its cost is measured in lost cumulative return rather than in a single incorrect output. Zhang et al. [16] formalise this setting as the **State-Adversarial Markov Decision Process (SA-MDP)**. It is defined by the tuple $(\mathcal{S}, \mathcal{A}, B, R, p, \gamma)$, where $\mathcal{S}$, $\mathcal{A}$, R , p and γ are the state space, action space, reward function, transition probability and discount factor of an ordinary MDP, and $B(s) \subseteq \mathcal{S}$ is the set of perturbations the adversary may apply at state s , commonly an ℓ_p -norm ball. The adversary is a deterministic, stationary and Markovian function $v : \mathcal{S} \rightarrow \mathcal{S}$ with $v(s) \in B(s)$. The agent selects its action from the perturbed observation, $a \sim \pi(\cdot | v(s))$, but the environment evolves from the true state s , so that

$$\tilde{V}_{\pi \circ v}(s) = \sum_{a \in \mathcal{A}} \pi(a | v(s)) \sum_{s' \in \mathcal{S}} p(s' | s, a) \left[R(s, a, s') + \gamma \tilde{V}_{\pi \circ v}(s') \right], \quad (2.5)$$

and the adversary corrupts only what the agent perceives, never the dynamics or the reward it receives [16].

The central result for attack design is that, for a *fixed* policy π , finding the optimal adversary is itself a standard reinforcement learning problem. Zhang et al. show (Lemma 1) that there exists an ordinary MDP whose actions are the perturbed observations presented to the agent and whose reward is the negative of the agent’s reward, and that the optimal policy of this MDP is the optimal adversary v^* . The optimal adversary can therefore be learned by an ordinary reinforcement learning algorithm that treats the victim as part of its environment, without differentiating the victim, and because it maximises an accumulated reward it takes into account how each perturbation shapes the states that follow. The framework also explains why robustness is hard to obtain: under the optimal adversary an optimal Markovian policy need not exist (Theorem 4), and the loss of value is bounded by how far the policy’s action distribution can be moved within the perturbation set (Theorem 5),

$$\max_{s \in \mathcal{S}} \{V_\pi(s) - \tilde{V}_{\pi \circ v^*}(s)\} \leq \alpha \max_{s \in \mathcal{S}} \max_{\hat{s} \in B(s)} D_{TV}(\pi(\cdot | s), \pi(\cdot | \hat{s})), \quad (2.6)$$

where α is a state-independent constant and D_{TV} the total variation distance. This bound motivates state-adversarial regularisers, instantiated as SA-PPO, SA-DDPG and SA-DQN, of which SA-DDPG is the closest to the MADDPG family. The same work proposes two attacks that do not need the victim’s critic: the Maximal Action Difference (MAD) attack, which maximises the divergence between the policy’s action distributions on clean and perturbed states, and the Robust Sarsa attack, which first learns an estimate of the victim’s value function and then uses it to guide a gradient perturbation [16].

Zhang et al. build on Lemma 1 in **Alternating Training with Learned Adversaries (ATLA)** [56], which trains an adversary against a fixed agent by policy gradient, then trains the agent against that adversary, and alternates. The learned adversary proves considerably stronger than the per-step attacks before it, lowering the victim’s reward well below critic-based, MAD, Robust Sarsa and snooping attacks, and it can drive an

agent in the opposite direction of its goal rather than merely halting it. Its weakness is cost: the adversary’s action space is the full observation space, which becomes hard to learn as the observation grows. **PA-AD** [57] addresses this by splitting the adversary into a director, which learns in which direction to push the policy, and an actor, which crafts the observation perturbation that realises it; it achieves strong attacks on Atari and MuJoCo tasks and, used in adversarial training, yields policies more robust than earlier defences.

These works establish that a learned adversary can be stronger than a gradient attack while knowing less about the victim, needing only to query its actions and to interact with the environment. They were, however, developed and evaluated for a single agent, and do not address settings in which several agents are perturbed at once. The attack of this thesis takes the learned adversary of the SA-MDP as its basis, applied to a frozen victim whose policies are fixed, which is exactly the condition of Lemma 1.

2.4.3 Strategically-timed attacks

Gradient attacks on reinforcement learning agents are usually applied at every time step. Lin et al. [58] argue that this ignores the sequential nature of the problem, since perturbations at different moments are not equally effective and an attack at every step is also more likely to be detected. Their **strategically-timed attack** minimises the agent’s expected return subject to a limit on the number of time steps at which a perturbation is applied. Because solving this constrained problem exactly is intractable, they decide when to attack with a heuristic, the gap between the probabilities the policy assigns to its most and least preferred actions (from a softmax of the Q-values for value-based agents), and attack only when this gap exceeds a threshold. At the chosen steps, a Carlini and Wagner attack [15] drives the agent to its originally least preferred action. On agents trained with DQN and A3C in five Atari games, the attack reduced the reward about as much as attacking at every step while acting on only about a quarter of the steps [58]. The same work introduces the enchanting attack, which plans a sequence of perturbations with a learned video prediction model to lure the agent to a chosen target state.

The result shows that where an attack is spent can matter as much as how often it is applied. The method has three limitations that bear on this thesis. Its threshold sets the attack rate only indirectly, so a requested budget is not guaranteed. Its timing signal is read from the victim’s own policy, which requires access to the policy’s outputs. And it was studied for single agents in games. The timing mechanism of this thesis addresses the first two by calibrating its threshold online and reading its signal from the network rather than from the victim (Section 3.4.3).

2.4.4 Multi-agent and joint attacks

State-Action Joint Attacks (SAJA) [18] start from the observation that earlier work attacks either the states or the actions of a multi-agent system in isolation. SAJA is a white-box attack on multi-agent actor-critic algorithms with a centralised critic, and it operates in

two phases. In the first, the observations of the victim agents are perturbed by iterated sign-gradient ascent within an ℓ_∞ ball, using both the actor and the critic: the actions a' that the policies produce from the perturbed observations are scored by the centralised critic, and the loss combines the negative critic value $-Q(s^{\text{adv}}, a')$ with an action-difference term $\|a' - a^{(0)}\|_2$. In the second phase the critic is used again to perturb the actions themselves. The action-difference term is justified by a bound showing that the loss of value grows with the multi-agent action difference (Theorem 2). Evaluated on MADDPG and FACMAC in the Multi-Agent Particle Environment with one to three victim agents, SAJA reduces the reward by up to 9.90%, bypasses the M3DDPG, PA-AD and ATLA defences, and outperforms the corresponding state-only and action-only attacks [18].

Other multi-agent attacks change the threat model rather than the objective. Attacks on the action space perturb or replace what agents do: SAJA’s action phase applies gradient ascent on the critic to the actions directly [18]. The Wolfpack attack controls adversarial agents that target one agent and then those that come to its aid, reducing rewards by up to 51.8%, severely affecting SMAC scenarios, and proving far more effective than random or single-agent attacks [59]. Adversarial Minority Influence (AMI) lets a minority of adversarial agents sway the majority towards failure, using a unilateral influence filter and a learned oracle of harmful target actions; it requires no access to the victims’ models and has been demonstrated on robot swarms, StarCraft II and multi-agent MuJoCo [34].

These works converge on one point: in multi-agent systems, perturbations chosen for their combined effect are more damaging than perturbations of individual agents. They differ in what the attacker must control. SAJA perturbs observations and actions and needs the victim’s centralised critic, while Wolfpack and AMI control entire agents. All were evaluated in games, particle environments and robotic tasks. The coordination mechanism of this thesis takes SAJA’s central idea, judging the perturbations of several agents by their joint effect, but keeps to an observation-only threat model and produces the joint perturbation with a learned adversary rather than with gradients of the victim’s critic (Section 3.4.2).

2.4.5 Other attack surfaces

Several further attack surfaces lie outside the threat model of this thesis and are noted only briefly. Systems that communicate explicitly can be attacked through their messages: an adversarial policy that modifies messages between agents can degrade team performance several-fold in simulation and by about a factor of two on physical robots [60], and adaptive attacks on communication persist even when messages are protected for privacy [61, 62]. Training-time attacks embed vulnerabilities in the learned policy. Poisoning attackers can coordinate through MADDPG itself to corrupt a federated model [63], and backdoor attacks implant triggers that make agents take attacker-chosen actions while behaving normally otherwise, reaching success rates close to 99% with little loss of performance on benign tasks [64–67]. Finally, attacks trained on randomised source environments

transfer to unseen target policies, and sometimes outperform white-box attacks on the target, which makes black-box attacks feasible without knowledge of the victim [68].

2.4.6 Summary of attacks

Tables 2.1 and 2.2 summarise the attacks reviewed in this section.

Table 2.1: Adversarial attacks on MADDPG and related learners: observation, action and communication space.

Attack domain	Attack	Mechanism	Reported efficacy
Observation space	FGSM/PGD	Single-step or iterative gradient perturbation of observations	Limited against MADDPG (unreliable critic gradients)
	Learned (ATLA)	Adversary trained by RL against a fixed agent	Stronger than MAD, Robust Sarsa and critic-based attacks
	PA-AD	Director learns policy perturbation; actor realises it in the observation	Strong attacks on Atari and MuJoCo
	Strategically timed	Attacks only when the policy's action preference exceeds a threshold	Matches every-step attack at about 25% of steps
	SAJA	Two-phase gradient ascent on states then actions; $-Q$ plus action difference	Up to 9.90% reward drop; stronger than state- or action-only attacks
Action space	Direct action manipulation	Gradient ascent on the critic for action perturbations	Effective with partial control of actuators
	Wolfpack	Adversarial agents target an agent and its helpers	Up to 51.8% reward reduction; severe SMAC impact
	AMI	Unilateral influence filter and targeted adversarial oracle	Robot swarms, StarCraft II, MuJoCo; black-box
Communication space	Message hijacking	Intercept and modify inter-agent messages	Several-fold degradation in simulation; about 2× on robots
	Adaptive communication attacks	Learned, persistent disruption of coordination	Persist despite privacy protection

2.5 Adversarial Attacks on Graph Neural Networks

A separate body of work studies adversarial attacks on GNNs themselves. These attacks perturb the graph a GNN operates on, by adding, removing or rewiring edges or by

Table 2.2: Adversarial attacks on MADDPG and related learners: training-time and transfer attacks.

Attack domain	Attack	Mechanism	Reported efficacy
Poisoning and backdoor	Model poisoning (federated)	Attackers coordinate through MADDPG to craft poisoned updates	Outperforms baseline poisoning
	Backdoor attacks	Triggers embedded during training	Success rate close to 99%; little benign loss
	PNAct	Positive and negative action samples toggle unsafe behaviour	Increases safety violations while keeping reward
Transfer and black-box	Domain-randomised transfer	Train on diverse environments, apply to unseen targets	Transfers across policies; sometimes beats white-box
	AMI (black-box)	Learns attack policies by interaction, without model access	No model knowledge required

altering node features, in order to mislead node-, link- or graph-level predictions, either at test time or by poisoning the training graph [69, 70]. Representative examples are Nettack [71], which perturbs both the structure and the features around a target node, and Metattack [72], which poisons the graph structure using meta-gradients. In this thesis the GNN serves only as an encoder of the routing agents’ observations, and the attacks perturb those observations rather than the graph itself, so attacks of this kind are not pursued further.

2.6 Summary and Research Gap

The literature reviewed in this chapter supports three observations. First, multi-agent reinforcement learning, and MADDPG in particular, is a well-established approach to routing, often combined with GNN encoders for their ability to generalise across topologies, yet these controllers are evaluated only on the observations the network actually produces. Second, the attack literature agrees that attacks which take the structure of the problem into account are stronger than single-step attacks applied at every step: PGD is stronger than FGSM on classifiers, learned SA-MDP adversaries are stronger than per-step attacks on single agents, strategically-timed attacks match the damage of every-step attacks at a fraction of the cost, and joint multi-agent perturbations such as SAJA’s outperform perturbations of individual agents. Third, these stronger attacks were developed and evaluated in games, robotic tasks and particle environments, and the learned and timed attacks for a single agent, rather than in networked control systems.

This leaves a gap. No work reviewed here evaluates observation-space attacks against

a multi-agent routing controller, a setting whose path redundancy may absorb redirected traffic and in which many agents act in parallel. The only such evaluation in this research line applied FGSM, a single-step, per-agent attack, and found that it did little damage [TODO-colleague-fgsm]; by the argument of Madry et al., that failure is weak evidence of robustness. Whether the controller withstands an attacker that is stronger in principle, a learned SA-MDP adversary that coordinates its perturbation across agents, as SAJA suggests, and divides its effort across time, as PGD and strategically-timed attacks suggest, has not been tested. This thesis addresses that question, through the research questions set out in Section 1.2.

METHODOLOGY

This chapter describes how the research question of Section 1.2 was investigated. That question asks whether the apparent robustness of MADDPG routing controllers to FGSM survives a theoretically stronger observation-space attack, one that coordinates its perturbation across agents and divides it across time. It is broken down into four questions: whether dividing the attack across time buys more damage per attacked step (RQ1), whether coordinating the perturbation across agents increases the damage (RQ2), which architectural choices of the victim determine its vulnerability (RQ3), and whether the fraction of routing decisions an attack changes is a meaningful measure of the damage it does (RQ4).

The chapter moves from the broadest methodological decisions to the most concrete procedures. Section 3.1 sets out the research approach and the experimental design, and relates each question to the comparison that answers it. Section 3.2 describes the components inherited from earlier work, and Section 3.3 the victims under test. Section 3.4 defines the threat model and the attack. Sections 3.5 and 3.6 describe how the data were collected and analysed, and Section 3.7 how the validity and reliability of the results were protected. Section 3.8 states the delimitations and limitations of the design, and Section 3.9 summarises the methodology and leads into the results.

3.1 Research Approach and Design

3.1.1 Research approach

The research questions are comparative and concern magnitudes: how much packet delivery an attack costs the victim, and whether one attack costs it more than another. The study is therefore quantitative, and its data are numerical measurements of packet delivery, routing decisions and attack timing.

It is conducted as a controlled experiment in simulation, for three reasons. First, the victims exist as policies trained in the simulator of the earlier work [17], and the FGSM baseline was measured in the same simulator, so a like-for-like comparison with that baseline requires the same environment. Second, a simulator can replay exactly the same

traffic under different attacks. Attacks can then be compared episode by episode rather than through averages over different traffic, which removes the largest source of variance in the comparison (Section 3.6.2). Third, corrupting the telemetry of a routing controller on an operational network could be done neither safely nor repeatably.

3.1.2 Experimental design

The study follows a factorial design with a control. Four factors are varied:

- **Victim variant:** the seven MADDPG variants of Section 3.3.
- **Attack scope:** *independent*, in which each agent’s perturbation is chosen from that agent’s own observation, or *coordinated*, in which the perturbations of all compromised agents are chosen together (Section 3.4.2).
- **Timing budget:** the fraction b of steps on which the attack may act, set to 0.10, 0.15, 0.20 or 0.25, or no budget, in which case the attack acts on every step (Section 3.4.3).
- **Domain constraint:** the *FGSM-parity* constraint of the baseline or the stricter *full* constraint (Section 3.4.1).

One adversary is trained for each combination of variant, scope and domain constraint, 28 in total, and each is evaluated at the four budgets and without a budget, giving 140 evaluated configurations. The budget is varied at evaluation rather than in training, so that the budgets are compared on the same trained adversary (Section 3.5.2). Everything else is held fixed across configurations: the perturbation radius, the set of compromised agents, the topology, the traffic regime and its load, the absence of link failures, and the traffic seed.

Each configuration is run as three arms on identical traffic: a clean arm without perturbation, a random control, and the attack. The random control is the control condition of the experiment. It shares the attack’s admissible set and timing gate but chooses its perturbation at random, so the difference between the two arms is attributable to the direction the adversary learned. The FGSM baseline, evaluated in the earlier study in the same simulator and on the same traffic, is the external reference. Table 3.1 relates each research question to the comparison that answers it, the measure used, and where the result is reported.

3.2 Starting Point and Inherited Components

This work is the third stage of a line of research. Because the research question is comparative, it depends on the earlier stages remaining unchanged, so this section states which components are inherited and which are contributed here.

Table 3.1: Research questions and the comparisons that answer them. The measures are defined in Section 3.6.

Question	Comparison	Measure	Reported in
Central	Learned adversary against FGSM, under the FGSM-parity constraint, on the same episodes	Δ^{FGSM} , Eq. (3.19)	Section 4.3
RQ1	Each timing budget against no budget, for the same trained adversary	Δ^{adv} and Δ^{adv} per unit of realised attack rate	Sections 4.2 and 4.4
RQ2	Coordinated against independent adversary, for the same variant, constraint and budget	Δ^{coord} , Eq. (3.17)	Sections 4.2.5 and 4.4.4
RQ3	The seven victim variants against one another	Δ^{adv} by variant	Chapter 4
RQ4	Routing decisions changed against delivery lost	Flip rate, Eq. (3.15), against Δ^{adv}	Sections 4.2.2 and 4.4.2
—	FGSM-parity against full domain constraint	Δ^{clamp} , Eq. (3.18)	Section 4.5

3.2.1 The routing system under test

The victim is a multi-agent routing controller developed in earlier work within the same research line [17]. It applies Multi-Agent Deep Deterministic Policy Gradient (MADDPG) to adaptive packet routing under the paradigm of centralised training with decentralised execution (CTDE) [5]: each agent i holds an actor π_{θ_i} that conditions only on its own local observation, while its critic Q_{ϕ_i} may, during training, observe the joint state and the actions of every other agent. Conditioning the critic on the joint action is what stabilises learning, since from any single agent’s perspective the environment would otherwise appear non-stationary as the other agents’ policies evolve [5]. After training the critics are discarded and each agent acts on local information alone.

The network is a service-provider topology of 86 nodes, 65 switches and 21 traffic endpoints, joined by 106 links, carrying flow-based traffic. The learning agents are its $M = 14$ distribution switches, the ingress routers to which the endpoints attach; the remaining switches forward traffic without a learned policy. Each agent observes a 94-dimensional vector: the available bandwidth to each of up to six neighbours (zero-padded), its queue depth, the diversity of destinations in its queue and the normalised time step, a presence flag for each of the $n_d = 21$ destinations, the mean, maximum and variance of its adjacent link utilisations, and, for every destination, the bottleneck utilisation of each of its $K = 3$ pre-computed candidate paths.¹ Every feature lies in $[0, 1]$. The agent’s action scores the K candidate paths of each destination, and the path with the highest score is taken. The trained system is thus a distributed controller in which no agent holds a global view, yet the paths chosen collectively determine end-to-end delivery. That prior work

¹The environment assembles a slightly longer vector and truncates it to 94 entries, which drops the last two path utilisations and a final hops-remaining feature.

also investigated critic design and graph-based state encoding, which is the origin of the variant family of Section 3.3.

This thesis does not modify the routing controller. The trained agents are treated as a fixed system under test: their parameters are frozen, and the attacks interact with them only through their observations and actions. The FGSM baseline additionally differentiates the agents' actors with respect to their inputs; the learned adversary of this thesis never does.

3.2.2 The FGSM baseline

The second stage established the adversarial setting. A single-step attack based on the Fast Gradient Sign Method (FGSM) [13] was developed against these agents in a preceding dissertation in the same line of work [TODO-colleague-fgsm]. It is a gradient-based attack, implemented independently of the learned adversary of this thesis. Applied to the routing controller, the victim is each agent's trained actor and the input is that agent's local observation. The objective weights the actor's score for every candidate path by a sigmoid of that path's bottleneck utilisation, read from the observation, so that ascending it steers the policy towards congested paths rather than inducing a misclassification. For the GNN variants the gradient is taken through the encoder, with the other agents' observations set to zero. The perturbation is applied to every agent at every time step. The threat model it fixed is *observation-space* rather than *actuation-space*: the adversary corrupts what the agents perceive, modelling a compromised telemetry or monitoring plane, while the network itself continues to evolve according to its true state.

That stage contributed not only an attack but an evaluation protocol: the stressed traffic regime, the frozen-victim assumption, a random control, and the metrics by which an attack is judged. This thesis retains all four.

3.2.3 The learned-adversary scaffold

The attack of this thesis is built on an existing implementation, provided as the starting point for this project, of the learned observation adversary of the State-Adversarial MDP of Zhang et al. [16] (Section 2.4.2). It trains a network against the frozen victim to perturb the agents' observations, learning from the damage its perturbations cause, and plugs into the same evaluation loop as the FGSM baseline. This work uses it as the vehicle for its two mechanisms: the SAJA-inspired coordination and the PGD-inspired timing described in Section 3.4 were built into it, together with the random control used to score it.

3.2.4 Contribution of this thesis

As Section 1.2 explains, the FGSM baseline's limited effect on delivery admits two readings: either the controller is genuinely robust, or a single-step, per-agent adversary is too weak for its failure to count as evidence of robustness. Separating the two requires an adversary without those two limitations, compared with the baseline on identical episodes and within

the same admissible set. This thesis contributes that adversary, by adding a coordination mechanism and a timing mechanism to the implementation of Section 3.2.3 (Section 3.4), together with the random control and the evaluation that score it. Such an adversary is theoretically stronger than FGSM (Section 3.4.4), so if it does no more damage than the baseline, the result strengthens the case that the controller is robust rather than that the baseline was too weak.

Because the purpose is comparison rather than a new attack in isolation, compatibility with the earlier stage is a design constraint rather than a convenience. The attack operates on the same frozen agents, within the same ℓ_∞ ball, and is scored under the same evaluation protocol as the FGSM baseline. Any difference in measured damage is then attributable to the attack itself, rather than to a change in the adversary’s capabilities or in the conditions of measurement. The domain constraint is the one respect in which the evaluation goes beyond this: the attack is evaluated under the baseline’s constraint and additionally under a stricter one, and only results under the former are directly comparable with the FGSM study (Section 3.4.1).

3.3 Victim Architectures Under Test

The attacks are evaluated against the family of trained variants inherited from the base system [17] rather than against a single model, so that conclusions about the routing task can be separated from conclusions about one particular network design, which is what RQ3 requires. The trained weights of every variant are used as they were delivered by the earlier work, which is also what makes the comparison with the FGSM study possible. All variants share the same actor: a multilayer perceptron with hidden layers of 256 and 128 rectified units and a sigmoid output, mapping the local observation to n_d blocks of K path scores, with the executed route given by the arg max within each block. They differ along three binary axes, described below.

3.3.1 Central (CC) versus local (LC) critic

The two critic domains differ in what the value function is allowed to observe during training:

$$\underbrace{Q_{\phi^i}(\mathbf{g}, a_1, \dots, a_M)}_{\text{CC: } |E|+M+M|\mathcal{A}|} \quad \text{versus} \quad \underbrace{Q_{\phi^i}(s_i^{\text{local}}, a_i)}_{\text{LC: } d_{\text{obs}}+|\mathcal{A}|}, \quad (3.1)$$

with $d_{\text{obs}} = 94$ the per-agent observation dimension and $|\mathcal{A}| = n_d K = 63$ the per-agent action dimension. The central critic follows the canonical MADDPG construction [5] in conditioning on the joint action, which removes the non-stationarity each agent would otherwise face as its peers’ policies change. Instead of every agent’s observation, it receives a compact global state $\mathbf{g}$: the available bandwidth on each of the $|E| = 106$ links and the most common destination in each agent’s queue. Its input still grows linearly in the

number of agents. The local critic reduces the scheme to independent actor–critic learners; it forfeits that guarantee but is far cheaper and its size is independent of the agent count.

Neither critic is used by the attacks of this thesis: the critics are discarded after training, and every variant is attacked by the same learned adversary. The distinction matters because it shapes how the victims’ policies were trained, which Chapter 4 draws on when interpreting the effect of coordination.

3.3.2 Simple versus duelling value head

The simple head passes the concatenated state and action through two rectified hidden layers and emits a scalar. The duelling head replaces the output layer with two parallel streams whose outputs are summed, $Q(s, a) = V(h) + A(h)$.

One caveat should be stated. In the original duelling formulation [30] the advantage stream emits one value *per action*, and subtracting the mean advantage makes the split identifiable. Here the action is an *input* to the critic rather than an output index, so both streams emit scalars and no such constraint is applied: any constant may be shifted between V and A without changing Q . The duelling head is thus an extra parallel projection rather than a value–advantage separation, and differences it produces should be attributed on that basis.

3.3.3 With and without GNN encoding

The GNN variants insert a shared encoder between the environment and the actor. Each agent is a graph node whose features are its full observation. Because the agents are a strict subset of the network’s nodes, the graph is the full physical topology: the 72 remaining nodes, the non-agent switches and the traffic endpoints, are included as relay nodes with zero features. Two GraphConv layers, with a hidden width of 64, aggregate neighbour information under a residual connection:

$$\tilde{\mathbf{X}} = \text{ReLU}\left(\text{GraphConv}_2(\text{ReLU}(\text{GraphConv}_1(\mathbf{X}, E)), E) + \mathbf{X}\right). \quad (3.2)$$

The encoder is dimension-preserving, so it is a drop-in pre-processor and the actor is unchanged between variants, and the residual connection makes it a near-identity transform at initialisation. The relay nodes act purely as message-passing intermediaries, so information travels at most two hops through the topology.

The consequence for the attacks is structural rather than one of degree: without the encoder an agent’s policy reads its own observation alone, whereas with it a perturbation applied to one agent also reaches the encoded state of any agent within two hops. In the independent scope the perturbations of nearby agents are therefore not fully independent in their effect. The learned adversary attacks these variants exactly as it attacks the others, since it needs no gradient path through the encoder.

Table 3.2: MADDPG victim variants under test, named <critic domain>-<value head>[-GNN].

Variant	Critic	Value head	GNN
CC-Simple	Central	Simple	—
CC-Duelling	Central	Duelling	—
CC-Simple-GNN	Central	Simple	Yes
CC-Duelling-GNN	Central	Duelling	Yes
LC-Simple	Local	Simple	—
LC-Duelling	Local	Duelling	—
LC-Duelling-GNN	Local	Duelling	Yes

3.3.4 Variant matrix

Table 3.2 lists the seven variants. CC-Simple is the canonical MADDPG formulation with no additional architectural machinery, and each other variant changes one or two of the three axes relative to it. LC-Simple-GNN, the eighth combination, is omitted, as the local-critic value-head comparison is already covered by LC-Simple/LC-Duelling and the GNN comparison by LC-Duelling/LC-Duelling-GNN.

3.4 Coordinated and Strategically-Timed Attacks on MADDPG Routing Agents

The attack of this thesis is a learned observation adversary in the sense of the State-Adversarial MDP [16], trained against the frozen victim, to which two mechanisms are added. They go beyond the FGSM baseline in the two respects identified in Section 3.2.4:

- **(A) Coordinated multi-agent perturbation, inspired by SAJA:** the perturbations of all compromised agents are produced together, from their joint observation, and judged by their combined effect on the network, as SAJA judges victim perturbations through the value of the joint action they induce [18]. Several compromised agents can then be steered towards the same shared link.
- **(B) Strategically-timed, critical-state attack, inspired by PGD:** the attack is divided across time, as PGD divides its budget into smaller steps [14]. A budget limits how often the adversary may act, and a critical-state score decides when it does, so that the attack is spent at the moments when the network’s redundancy is thinnest.

The threat model is set out first, since it defines what the attacker is assumed to know and to control. The two mechanisms and the learned adversary that realises them follow.

3.4.1 Threat model and admissible perturbation set

The threat model is based deliberately on the one used for the FGSM baseline, so that a difference in damage can be attributed to the attack rather than to a change in what

the attacker is allowed to do. It differs in what the attacker needs to know, which is summarised in Table 3.3, and the attack is additionally evaluated under a stricter domain constraint.

Victim and compromised agents

The victim policies $\{\pi_{\theta_i}\}$ are *frozen*: their parameters are fixed throughout, and no victim parameter is ever updated, so the attack measures the robustness of a fixed trained policy rather than the outcome of an adversarial training game. Let $C \subseteq \{1, \dots, M\}$ denote the set of compromised agents and $N = |C|$ its size. In every experiment of this thesis all agents are compromised, so C contains the $M = 14$ distribution switches and $N = 14$.

Capability

The attacker controls the observations of the compromised agents and nothing else. Whenever it acts, it replaces each compromised agent’s observation with a perturbed one, modelling a compromised telemetry or monitoring plane, and the agents choose their actions from the perturbed observations. It cannot alter the agents’ actions, their parameters, the traffic, the links or the rewards, and the network continues to evolve from its true state. When the timing mechanism of Section 3.4.3 is used, the attacker acts on all compromised agents at the same steps and leaves the observations untouched at the others. For agent i at time t the admissible set is

$$\mathcal{S}(s_{i,t}, \varepsilon) = \left\{ \hat{s} \mid \|\hat{s} - s_{i,t}\|_{\infty} \leq \varepsilon, \hat{s} \in \mathcal{D} \right\}, \quad (3.3)$$

with $\varepsilon = 0.30$, where $\mathcal{D}$ is a domain constraint and $\Pi_{\mathcal{S}}$ denotes the projection onto $\mathcal{S}$: an ℓ_{∞} clip about the clean observation followed by a clip of the constrained features onto $[0, 1]$.

Two domain constraints are used. The *FGSM-parity* constraint is the one of the FGSM baseline: it clips the first four features of each observation, which are four of the six available-bandwidth features, and leaves every other feature free within the ε -ball. It is needed for the comparison with the baseline, but it allows most features to leave the range $[0, 1]$ in which the environment produces them, so a perturbed observation can report values that no real telemetry would, such as a negative link utilisation. The *full* constraint rules this out by clipping every feature onto $[0, 1]$, and so models an attacker whose falsified telemetry stays within the valid range. It gives a strict subset of the parity set, and its consequences are discussed in Section 3.4.4. The random control of Section 3.5.3 always uses the same projection as the attack it controls, since comparing an attack with a control drawn from a different set would confound the perturbation direction with the size of the set it is drawn from.

Table 3.3: What each attack needs to know or access. The learned adversary’s requirements differ between training and attack time; the timing gate’s requirement applies only when a timing budget is used.

Knowledge or access	FGSM	Learned adversary	
	baseline	training	attack time
Victim parameters and gradients	Yes	No	No
Querying the victim’s actions	Yes	Yes	No
Clean observations of the compromised agents	Yes	Yes	Yes
Write access to those observations	Yes	Yes	Yes
A simulator of the network and its traffic	No	Yes	No
Per-step count of dropped and forwarded packets	No	Yes	No
Utilisation of every link (timing gate)	No	Yes	Yes
Available bandwidth of every link and each agent’s queue destinations (coordinated critic)	No	Yes	No

Knowledge

The FGSM baseline is a white-box attack: at every step it differentiates each agent’s actor, and for the GNN variants the encoder, with respect to that agent’s observation. The learned adversary never needs the victim’s parameters or gradients. During training it needs to run the victim inside a faithful simulator of the network, querying the victim’s actions on perturbed observations and receiving, at every step, the number of packets dropped and forwarded, from which its reward is computed. In the coordinated scope its critic also reads the available bandwidth of every link and the most common destination in each agent’s queue. At attack time the trained adversary needs only the clean observations of the compromised agents: in the independent scope each agent’s own observation, and in the coordinated scope all fourteen at once. Whenever a timing budget is used, however, both in training and at attack time, the timing gate reads the utilisation of every link in the network to score the current step. The attack is therefore less demanding than FGSM in what it must know about the victim, but more demanding in what it must know about the network: it assumes a faithful simulator for training and, when timed, network-wide link telemetry during the attack.

3.4.2 (A) Coordinated multi-agent perturbation

The coordination mechanism is inspired by SAJA [18] (Section 2.4.4). SAJA perturbs the observations of several victim agents at once and scores each candidate perturbation by the centralised critic’s value of the joint action the agents then take, so the perturbations are judged by their combined effect rather than one agent at a time. This subsection explains why that matters in a routing network and how the learned adversary implements it.

Separability of the per-agent attack

The FGSM baseline, and any attack that chooses each agent’s perturbation independently, solves N decoupled problems: for each $i \in C$,

$$\eta_{i,t}^* = \arg \max_{\eta: s_{i,t} + \eta \in S} \mathcal{L}_i(s_{i,t} + \eta), \quad (3.4)$$

where $\mathcal{L}_i$ depends only on agent i ’s own observation and policy. The joint perturbation obtained this way is the concatenation of N independent solutions, and the implied joint objective is *separable*,

$$\mathcal{J}_{\text{sep}}(\hat{\mathbf{s}}_t) = \sum_{i \in C} \mathcal{L}_i(\hat{s}_{i,t}), \quad (3.5)$$

so no term couples two agents. This is a structural limitation rather than a matter of attack strength: a separable objective cannot express a preference for outcomes that depend on *which* agents are steered onto the *same* resource, because the value of steering agent i onto a link is independent of whether agent j was steered onto it as well. In a topology whose robustness comes from K -path redundancy, the damaging configurations are precisely those in which several redirected flows converge on one shared link, and these lie outside the reach of Equation (3.5).

Joint perturbation

The coordinated adversary therefore chooses the perturbations of all compromised agents together. Writing $\mathbf{s}_t = (s_{1,t}, \dots, s_{N,t})$ for the concatenated clean observations of the compromised agents, a single actor v_ω maps the joint observation to one joint perturbation direction, and the joint observation presented to the victim is

$$\hat{\mathbf{s}}_t = \Pi_S(\mathbf{s}_t + \varepsilon v_\omega(\mathbf{s}_t)), \quad v_\omega(\mathbf{s}_t) \in [-1, 1]^{Nd_{\text{obs}}}, \quad (3.6)$$

where the projection is applied to each agent’s block separately, so that each agent’s perturbed observation lies in its own admissible set. Each agent then receives its own block of $\hat{\mathbf{s}}_t$, and all compromised agents act on one coordinated decision rather than on N independent ones. Where SAJA scores a joint perturbation with the victim’s own centralised critic, the coordinated adversary learns its own critic, which scores the joint perturbation by the damage it is expected to cause and is additionally given the available bandwidth of every link and each agent’s most common queued destination. Learning its own critic keeps the attack independent of the victim’s internals and makes it applicable to the local-critic variants, which have no joint critic to use. Because the actor’s input and the critic’s evaluation are joint, and the reward is the damage to the whole network, the adversary is not restricted to separable objectives of the form of Equation (3.5). How it is trained is described in Section 3.4.4.

3.4.3 (B) Strategically-timed attacks under an L_0 budget

Without a budget the adversary acts at every time step. A more realistic adversary is constrained in *how often* it can act, which converts the attack into a resource-allocation problem over the trajectory.

Dividing the attack in this way is how the design carries PGD’s central idea from the input into time. FGSM spends its entire budget in a single step of size ϵ , and the FGSM baseline did so at every time step for every agent. PGD instead divides its budget into a sequence of smaller steps of size α [14]. By analogy, rather than applying a full-strength perturbation at every step, the attack divides its effort across the trajectory and acts on only a fraction of the steps. The analogy concerns *when* the adversary acts, not how strongly: every step on which it does act is still bounded by the same ϵ . The attack is therefore subject to an L_0 budget: over a run of T consecutive decisions and for a budget fraction $b \in (0, 1]$,

$$\frac{1}{T} \sum_{t=1}^T \mathbf{1}[\text{attack at } t] \leq b. \quad (3.7)$$

The budget applies to the cumulative attack rate over a whole training or evaluation run, which spans many episodes, not to each episode separately.

Limiting how often an attack acts was first studied by Lin et al. [58], whose strategically-timed attack minimises an agent’s expected return subject to a cap of this kind on the number of perturbed steps, and from whom the name of this mechanism is taken (Section 2.4.3). The timing mechanism of this thesis shares that idea but differs from their attack in three respects. First, its critical-state score is read from the network, as the utilisation of its links, rather than from the victim’s policy, whose preference between its most and least favoured actions Lin et al. compare with a threshold. Second, its threshold is recalibrated online as a quantile and ties at the threshold are rationed, so that the realised attack rate matches the requested budget, whereas a fixed threshold controls the rate only indirectly. Third, the perturbation applied at the chosen steps is produced by the learned adversary for all compromised agents at once, rather than by a Carlini and Wagner attack that drives a single agent towards its least preferred action.

Spending this budget at random steps would spend much of it at moments when the network has ample spare capacity. The attack instead spends it at moments of high leverage, identified by a *critical-state score*, the maximum link utilisation across the topology,

$$c_t = \max_{(u,v) \in E} \rho_t(u, v), \quad (3.8)$$

where $\rho_t(u, v)$ is the utilisation of link (u, v) at time t . This single quantity responds both to congestion onset, which raises it directly, and to the loss of a link, which raises it indirectly by concentrating traffic onto the remaining paths, so the score is elevated exactly when redundancy is thinnest. Computing it requires the utilisation of every link, which is the network-wide knowledge noted in Table 3.3.

Because the distribution of c_t is not known in advance, the trigger threshold is calibrated online. Let $\mathcal{H}_t$ be a sliding window of the $W = 1000$ most recent scores and

$$\theta_t = Q_{1-b}(\mathcal{H}_t), \quad (3.9)$$

where Q_{1-b} denotes the empirical $(1 - b)$ -quantile, computed from the window *before* c_t is inserted so that a step never influences its own decision. The window is not reset between episodes.

A threshold test of the form $c_t \geq \theta_t$ is not sufficient here, because the score distribution has an *atom*: a single value taken by a large share of steps. Link utilisation is accumulated from fixed-size flow reservations, so the score returns to exactly the same values again and again; the evaluation of Chapter 4 locates the atom that matters at the tightest budgets at $c_t = 0.90$ (Section 4.2.3). Whenever b is smaller than the fraction of steps scoring at or above an atom, the quantile falls on it, θ_t takes the atom's value, and every tied step passes the test. The realised attack rate is then that fraction rather than b : the constraint of Equation (3.7) is violated, and any two budgets below that fraction yield the same attack, so the budget ceases to be the independent variable the experiment varies. Preliminary runs with the plain threshold test showed exactly this: a requested budget of 0.10 realised an attack rate of about 0.21, and the budgets of 0.10 and 0.15 produced the same attack.

The ties are therefore *rationed*. Writing D_t for the number of gate decisions up to and including step t , and F_{t-1} for the number of attacks already made, the adversary acts at step t if

$$\text{fire}_t = \begin{cases} 1, & c_t > \theta_t, \\ 1, & c_t = \theta_t \text{ and } F_{t-1} + 1 \leq b D_t, \\ 0, & \text{otherwise,} \end{cases} \quad (3.10)$$

so that steps strictly above the threshold always fire, while tied steps are admitted only for as long as the cumulative rate stays within budget. The tie-break is deterministic rather than randomised: a random rule would introduce variance between the attacked and the control arm, which are intended to differ only in the *direction* of the perturbation, and would require a separate random stream so as not to disturb the seeded traffic sequence.

The first $H = 100$ decisions of a run only observe. They seed the window without attacking, since the quantile is not meaningful until the window holds enough scores. Those decisions still count towards D_t , so the allowance left unspent during the warm-up is recovered afterwards. Because steps strictly above the threshold fire regardless of the running count, the realised rate over a run settles close to b but can slightly exceed it. Since it is the realised rate, not the requested one, that a comparison across budgets actually varies, it is measured and reported alongside every result rather than assumed equal to b . The procedure is summarised in Algorithm 1.

Algorithm 1: Critical-state timing gate with rationed ties

Input: budget b ; window $\mathcal{H}$ of size W ; warm-up length H ; running counts D (decisions) and F (attacks made); network state at step t

Output: decision $fire \in \{\text{true}, \text{false}\}$

```

 $c_t \leftarrow \max_{(u,v) \in E} \rho_t(u, v)$  // critical-state score
 $D \leftarrow D + 1$ 
if  $|\mathcal{H}| < H$  then
  |  $fire \leftarrow \text{false}$  // warm-up: observe, do not act
else
  |  $\theta_t \leftarrow Q_{1-b}(\mathcal{H})$  // computed before inserting  $c_t$ 
  | if  $c_t > \theta_t$  then
  | |  $fire \leftarrow \text{true}$ 
  | else if  $c_t = \theta_t$  then
  | |  $fire \leftarrow (F + 1 \leq b D)$  // ration the tie
  | else
  | |  $fire \leftarrow \text{false}$ 
  | end
end
insert  $c_t$  into  $\mathcal{H}$  (ring buffer of size  $W$ )
if  $fire$  then  $F \leftarrow F + 1$ 
return  $fire$ 

```

The gate composes with either attack scope: it decides *whether* to act at a given step, one decision shared by all compromised agents, while the scope decides *how* the perturbation is produced once the decision to act has been taken. On skipped steps the agents receive their unmodified observations. When no budget is set there is no gate, and the adversary acts on every agent at every step.

3.4.4 Learned observation-space adversary

Sections 3.4.2 and 3.4.3 describe what the attack should do. It should perturb the compromised agents jointly, keep each perturbation within the admissible set, and divide its effort across time by acting on only a fraction of the steps. Both mechanisms are realised in a learned adversary, built on the implementation of Section 3.2.3.

The learned adversary comes from the State-Adversarial MDP of Zhang et al. [16] (Section 2.4.2). Because the victim is frozen, its policies are fixed, which is exactly the condition of Lemma 1: against a fixed policy, finding the optimal observation adversary is itself an ordinary MDP, and can therefore be approached with standard reinforcement learning. Such an adversary is trained here with DDPG, treating the frozen victim as part of its environment. Learning the perturbation from the damage it causes, rather than solving for it at every step against an immediate objective, lets the attack value a perturbation by its effect on later steps and on the network as a whole, and removes the

need for the victim’s gradients. The adversary is denoted v_ω , deliberately reusing the symbol of the SA-MDP adversary, with parameters ω .

Formulation

In the independent scope, at time t the adversary observes the clean observation $s_{i,t}$ of a compromised agent $i \in C$ and emits a perturbation direction $a_{i,t}^v = v_\omega(s_{i,t}) \in [-1, 1]^{d_{\text{obs}}}$, produced by a tanh output layer. The observation presented to the victim is

$$\hat{s}_{i,t} = \Pi_S(s_{i,t} + \varepsilon a_{i,t}^v), \quad (3.11)$$

so that every perturbation lies in the admissible set by construction. In the coordinated scope the same construction is applied to the joint observation, as in Equation (3.6). Lemma 1 takes the perturbed state itself as the adversary’s action; parameterising it instead as a bounded direction scaled by ε is an equivalent reparameterisation that keeps the actor’s output within the admissible ball without a penalty term. The victim acts on $\hat{s}_{i,t}$, while the network continues to evolve from the true state, as the SA-MDP requires.

Lemma 1 prescribes the negative of the victim’s reward as the adversary’s reward. The adversary of this thesis departs from this deliberately. The victim is trained on a shaped per-agent reward combining several utilisation-dependent terms, which is a training signal rather than the quantity whose degradation this work measures. The adversary is instead rewarded with the fraction of forwarding events at step t that end in a drop,

$$r_t = \frac{n_t^{\text{drop}}}{\max(1, n_t^{\text{fwd}})}, \quad (3.12)$$

where n_t^{fwd} counts packets forwarded by any agent at step t and n_t^{drop} those dropped. This aligns the adversary’s objective with packet delivery, the outcome measured in Section 3.6.1. It is a proxy rather than an exact match: because each packet contributes one forwarding event per hop, the denominator grows with path length, so the signal is diluted relative to the end-to-end packet delivery ratio on which the adversary is ultimately scored.

Training

The actor v_ω and a critic Q_ψ^v are multilayer perceptrons with two hidden layers of 256 ReLU units. The critic scores the perturbation that was *actually applied*. Because the projection in Equation (3.11) can truncate the actor’s output, the stored direction is the effective one, $\tilde{a}_{i,t}^v = (\hat{s}_{i,t} - s_{i,t})/\varepsilon$, rather than the raw output. Transitions $(s_{i,t}, \tilde{a}_{i,t}^v, r_t, s_{i,t+1})$ are stored in a replay buffer $\mathcal{R}$, and the critic is fitted to a one-step temporal-difference target computed with slowly updated target networks $v_{\omega'}$ and $Q_{\psi'}^v$,

$$\mathcal{L}(\psi) = \mathbb{E}_{\mathcal{R}} \left[(Q_\psi^v(s, \tilde{a}^v) - r - \gamma Q_{\psi'}^v(s', v_{\omega'}(s')))^2 \right], \quad (3.13)$$

while the actor ascends the critic’s estimate of the damage its perturbation causes,

$$\max_{\omega} \mathbb{E}_{\mathcal{R}} \left[Q_\psi^v(s, v_\omega(s)) \right]. \quad (3.14)$$

The episode is treated as a continuing task, so the target in Equation (3.13) is never truncated at the episode boundary. During training Gaussian noise $\xi \sim \mathcal{N}(0, \sigma^2 I)$ with $\sigma = 0.10$ is added to the actor's output and clipped back to $[-1, 1]$. The remaining settings are $\gamma = 0.95$, learning rates of 10^{-4} for the actor and 10^{-3} for the critic, a minibatch of $m = 256$, a buffer capacity of 2×10^5 transitions, and a soft target update with rate $\lambda = 5 \times 10^{-3}$. One gradient update is performed per environment step once $\mathcal{R}$ holds at least $n_{\text{warm}} = 2000$ transitions. How long each adversary is trained, and under which conditions, is given in Section 3.5.2. The procedure is summarised in Algorithm 2.

Algorithm 2: Training the learned observation-space adversary (independent scope)

Input: frozen victim $\{\pi_{\theta_i}\}$; radius ε ; timing budget b ; episodes E ; steps per episode

T_{ep}

Output: adversary actor v_ω

initialise v_ω, Q_ψ^v ; set $\omega' \leftarrow \omega, \psi' \leftarrow \psi$; empty replay buffer $\mathcal{R}$

for $\text{episode} = 1, \dots, E$ **do**

 reset the network and observe $\{s_{i,1}\}_{i \in C}$

for $t = 1, \dots, T_{\text{ep}}$ **do**

if the timing gate fires at t (Algorithm 1) **then**

foreach $i \in C$ **do**

$a_{i,t}^v \leftarrow \text{clip}(v_\omega(s_{i,t}) + \xi, -1, 1), \quad \xi \sim \mathcal{N}(0, \sigma^2 I)$

$\hat{s}_{i,t} \leftarrow \Pi_S(s_{i,t} + \varepsilon a_{i,t}^v); \quad \tilde{a}_{i,t}^v \leftarrow (\hat{s}_{i,t} - s_{i,t})/\varepsilon$

end

else

$\hat{s}_{i,t} \leftarrow s_{i,t}$ for every $i \in C$ // no attack, no transition

end

 the victim acts on $\{\hat{s}_{i,t}\}$; the network steps from the true state

$r_t \leftarrow n_t^{\text{drop}} / \max(1, n_t^{\text{fwd}})$

if the gate fired **then** store $(s_{i,t}, \tilde{a}_{i,t}^v, r_t, s_{i,t+1})$ in $\mathcal{R}$ for each $i \in C$

if $|\mathcal{R}| \geq \max(m, n_{\text{warm}})$ **then**

 sample a minibatch of size m from $\mathcal{R}$

 update ψ by Equation (3.13) and ω by Equation (3.14)

$\omega' \leftarrow \lambda \omega + (1 - \lambda) \omega'; \quad \psi' \leftarrow \lambda \psi + (1 - \lambda) \psi'$

end

end

end

Scope and timing

Both mechanisms apply to the learned adversary. In the *independent* scope a single actor, shared by all compromised agents, maps each agent's own observation to that agent's

perturbation. Every agent contributes its own transition, and all transitions share the network-wide reward r_t . Each perturbation is therefore still chosen from one agent’s view alone, as in the separable problem of Equation (3.4), but the shared reward trains the actor towards perturbations that degrade the network as a whole. In the *coordinated* scope of Section 3.4.2, the observations of all compromised agents are concatenated into one joint input. A single actor then emits one joint perturbation, and the critic is additionally conditioned on the available bandwidth of every link together with each agent’s most common queued destination. This yields one transition per step for the whole group.

The timing gate of Section 3.4.3 is applied during training as well as evaluation. On a step it skips, the victim receives clean observations and no transition is stored, but the gradient update of Algorithm 2 still runs. A tighter budget therefore reduces the number of fresh transitions without reducing the number of updates, and each stored transition is replayed correspondingly more often. The effect is compounded in the coordinated scope, which stores one transition per step rather than one per agent. At evaluation the actor is used deterministically, without exploration noise, and it neither queries the victim nor learns further.

Relation to the FGSM baseline

The learned adversary is theoretically stronger than the FGSM baseline. Under Lemma 1 the optimal SA-MDP adversary is the best of all deterministic perturbation policies within the admissible set, and FGSM, which computes its perturbation deterministically from the current observation, is one such policy. Within the same admissible set and acting on the same steps, an optimal learned adversary can therefore do no worse than FGSM with respect to its own objective, and it can additionally exploit what FGSM cannot: perturbations that pay off over later steps, coordination across agents, and timing. A trained adversary only approximates that optimum, since it is limited by its training, which is why the comparison of Section 4.3 is needed. If the theoretically stronger attacker does no more damage than FGSM, the result supports the robustness of the controller rather than the weakness of the baseline.

It also requires strictly *less* knowledge of the victim, as set out in Section 3.4.1: FGSM differentiates the victim with respect to its input observations, whereas the learned adversary never computes a gradient of the victim. In exchange it needs a faithful simulator of the network during training and, when timed, network-wide link utilisation during the attack.

Its admissible set depends on the domain constraint. Under the *FGSM-parity* constraint it perturbs within exactly the set of the FGSM baseline. Under the *full* constraint every feature is clipped onto $[0, 1]$, which yields a strict subset of that set. Measured on sampled on-policy observations, the full constraint removes roughly a third of the available perturbation range and binds on around four fifths of feature–step pairs. Nearly all of this loss comes from the lower limit of $[0, 1]$: over half of the observation vector sits at exactly

zero, where clipping removes the downward half of the ε ball. A separate adversary is trained under each constraint and always evaluated under the constraint it was trained with, since an actor’s direction is fitted to the projection it was trained under. Only the results under the FGSM-parity constraint are directly comparable with those of the FGSM study. The smaller set of the full constraint does not make its results a lower bound on those under the parity constraint: each has its own trained adversary, and Section 4.5 shows the two adversaries harming different victims.

3.5 Data Collection

3.5.1 Environment and traffic regime

All training and evaluation runs take place in the simulator of Section 3.2.1, in the same stressed traffic regime as the FGSM baseline: an offered load factor of 2.0 with a hotspot traffic matrix. Each flow is, with probability 0.75, sent from one of eight designated sources (the endpoints BS1–BS4 and MECS1–MECS4) to one of the two endpoints CS1 and CS2, and otherwise between two endpoints chosen uniformly. The regime is stressed on purpose: at nominal uniform load the network has ample spare capacity, so routing decisions, and therefore perturbations of them, barely affect delivery. No link failures are injected. The adversary’s training loop does not inject failures, so evaluating without them keeps every evaluation within the conditions its adversary was trained under. An episode lasts 256 steps, and the network is reset at the start of every episode.

3.5.2 Training the adversaries

One adversary was trained for each of the 28 combinations of victim variant, attack scope and domain constraint, with the settings of Section 3.4.4. Each was trained for 300 episodes of 256 steps against its frozen victim, with $\varepsilon = 0.30$, under the domain constraint it would later be evaluated under, and with the timing gate set to $b = 0.25$. That budget is the loosest in the sweep, so the tighter evaluation budgets act on a subset of the states encountered during training; evaluated without a budget, the adversary also acts on states it did not encounter. Training is not seeded, so repeated training runs would produce different adversaries, and each configuration was trained once (Section 3.7.3). Each trained adversary is stored with its actor weights, its perturbation radius, its scope and the domain constraint it was trained under.

3.5.3 Evaluation runs

Each trained adversary was evaluated at the four timing budgets and without a budget. Each of these 140 configurations is evaluated with three arms of 30 episodes each: a *clean* arm with no perturbation, a *random* control, and the *attack* under test. The random control draws the perturbation of every feature independently and uniformly from $[-\varepsilon, \varepsilon]$, applies

the same projection as the attack, and, when a timing budget is set, is gated by its own copy of the same timing gate. The attack arm uses the trained actor deterministically.

All three arms are run on identical episodes. The traffic sequence is generated from a common seed, 20240601, to which the random generators are reset at the start of every arm, so episode e of one arm and episode e of another differ only in the perturbation applied. The random control draws from a dedicated random generator so that it consumes no entropy from the traffic generator, which would otherwise desynchronise the arms and invalidate the pairing.

The FGSM baseline is not re-run. Its results are taken from the earlier study, which evaluated it on 15 episodes at the same load and traffic seed, without link failures, under the FGSM-parity constraint and at $\varepsilon = 0.30$. Because the traffic sequence is generated from the same seed, the first 15 of the 30 episodes of every evaluation are the episodes on which FGSM was evaluated. The FGSM study’s random control differs from the one used here: it perturbs every feature by $\pm\varepsilon$ with a random sign.

3.5.4 Recorded data

Each configuration produces one record, which stores:

- the configuration itself: victim variant, scope, perturbation radius, timing budget, load, number of link failures, traffic seed and domain constraint;
- for each arm, the packet delivery ratio of every episode;
- for the random control and the attack, the fraction of routing decisions changed (Section 3.6.1);
- for each gated arm, the timing gate’s counts: decisions taken, steps attacked, realised attack rate, warm-up decisions, and ties seen, admitted and rejected.

The FGSM-parity runs additionally record why the gate fired: for every attacked step, the case of Equation (3.10) that admitted it and the congestion band its score fell in, namely saturated ($c_t \geq 1.00$), near saturation ($0.90 \leq c_t < 1.00$), high congestion ($0.75 \leq c_t < 0.90$) or moderate ($c_t < 0.75$). This recording was added after the full-constraint runs had been made.

3.5.5 Implementation and platform

The simulator, the victims, the FGSM baseline and the learned adversary are implemented in Python, with PyTorch for the neural networks, PyTorch Geometric for the GNN encoder and NetworkX for the topology. The learned adversary exposes the same interface as the FGSM baseline, so both attacks are scored by the same evaluation loop. All runs were executed inside a Docker container on a university GPU server, and the tables and figures of Chapter 4 are computed directly from the stored records.

3.6 Data Analysis

3.6.1 Measures

The *outcome* of an episode is its end-to-end packet delivery ratio (PDR): the percentage of the packets injected during the episode that are delivered by its end. Packets still in transit when the episode ends count as not delivered. The PDR is computed in the same way in every arm and in the FGSM study.

The *decisions* an attack changes are measured by the action flip rate, the fraction of per-agent, per-destination $\arg \max$ path choices that differ between the clean and the perturbed observations at the same time step,

$$\text{flip} = \frac{1}{|C| n_d T_{\text{ep}}} \sum_t \sum_{i \in C} \sum_d \mathbf{1} \left[\arg \max_k \pi_{\theta_i}(\hat{s}_{i,t})_{d,k} \neq \arg \max_k \pi_{\theta_i}(s_{i,t})_{d,k} \right], \quad (3.15)$$

averaged over the episodes of an arm and counted over every step, including the steps a timing gate skipped, on which no decision changes. The two are reported separately because they can diverge. In a topology with K -path redundancy, a high flip rate accompanied by an unchanged PDR indicates that the network absorbed the redirected flows rather than that the attack succeeded, and reporting a flip rate as though it were a measure of damage would conflate the two. Whether they diverge in practice is RQ4.

The *timing* of an attack is described by its realised attack rate, the fraction of gate decisions on which it attacked, F_T/D_T , and by the composition of its triggers, the share of attacked steps admitted by each case of Equation (3.10) in each congestion band.

3.6.2 Adversarial gap and confidence intervals

Any perturbation of the observations disturbs the controller, whether or not its direction was chosen adversarially, so the drop in PDR relative to the clean arm does not isolate what the attacker's choice of perturbation contributes. The primary measure is therefore the *adversarial gap*, the mean paired per-episode difference in PDR between the random control and the attack,

$$\Delta^{\text{adv}} = \frac{1}{n} \sum_{e=1}^n \left(\text{PDR}_e^{\text{rand}} - \text{PDR}_e^{\text{atk}} \right), \quad (3.16)$$

in percentage points, with $n = 30$. A positive gap means the attack cost the victim more delivery than the random control did. It is reported with a paired 95% confidence interval, $\Delta^{\text{adv}} \pm t_{0.975, n-1} s_d / \sqrt{n}$, where s_d is the standard deviation of the per-episode differences and $t_{0.975, 29} = 2.045$. Differencing within episode pairs removes the episode-to-episode variation in offered traffic, which would otherwise dominate the comparison. The drop from the clean arm to the attack is reported alongside the gap, but is not used as the measure of attack strength.

A result is treated as evidence of an effect only when its confidence interval excludes zero, which is equivalent to a two-sided paired t -test at the 5% level. An interval that spans

zero is reported as a null result, not as evidence of robustness, since each configuration can only resolve effects larger than its interval's half-width. The interval assumes that the per-episode differences are independent and approximately normally distributed. Every episode starts from a reset network with fresh traffic, but the timing gate's window and counters carry over from one episode to the next, so the episodes of a gated arm are not strictly independent. No formal test of normality was applied; with 30 pairs the t -interval is robust to moderate departures from it.

3.6.3 Derived comparisons

Three further comparisons are paired in the same way and carry a confidence interval of the same form.

The *effect of coordination* compares the coordinated and independent adversaries trained against the same variant, under the same constraint and at the same budget. Neither the clean arm nor the random control depends on the adversary, so both are the same in the two scopes, and the difference between the two adversarial gaps reduces to the per-episode difference between the two attack arms,

$$\Delta^{\text{coord}} = \Delta_{\text{coord}}^{\text{adv}} - \Delta_{\text{ind}}^{\text{adv}} = \frac{1}{n} \sum_{e=1}^n \left(\text{PDR}_e^{\text{atk,ind}} - \text{PDR}_e^{\text{atk,coord}} \right). \quad (3.17)$$

A positive value means coordination increased the damage.

The *effect of the domain constraint* compares the adversaries trained under the two constraints. Their random controls draw from different sets and therefore differ, so the comparison is the paired difference between the two adversarial gaps,

$$\Delta^{\text{clamp}} = \Delta_{\text{parity}}^{\text{adv}} - \Delta_{\text{full}}^{\text{adv}} = \frac{1}{n} \sum_{e=1}^n \left[\left(\text{PDR}_e^{\text{rnd,parity}} - \text{PDR}_e^{\text{atk,parity}} \right) - \left(\text{PDR}_e^{\text{rnd,full}} - \text{PDR}_e^{\text{atk,full}} \right) \right]. \quad (3.18)$$

A positive value means the adversary trained under the FGSM-parity constraint did more damage, relative to its own random control, than the one trained under the full constraint.

The *comparison with FGSM* uses only the FGSM-parity runs, in which both attacks perturb within the same admissible set. The two studies used different random controls, so their adversarial gaps are measured against different references and are not differenced. The attack arms themselves are compared instead, on the 15 episodes the two evaluations share,

$$\Delta^{\text{FGSM}} = \frac{1}{15} \sum_{e=1}^{15} \left(\text{PDR}_e^{\text{atk,FGSM}} - \text{PDR}_e^{\text{atk,learned}} \right), \quad (3.19)$$

with a paired 95% confidence interval using $t_{0.975,14} = 2.145$. A positive value means the learned adversary did more damage than FGSM. For reference, FGSM's own adversarial gap against its own random control is reported on the same 15 episodes.

The pairing of all three comparisons rests on the arms concerned running on identical traffic, which is checked on the records before each comparison is made (Chapter 4).

3.6.4 Damage per attacked step

RQ1 asks whether a limited budget buys more damage *per attacked step*. A gated attack acts on fewer steps than an ungated one, so its total damage is expected to be lower, and the comparison is made after normalising by the realised attack rate: the adversarial gap divided by the realised rate gives the damage per unit of attack rate, and a tighter budget is more efficient when this quantity is larger than when attacking every step. In the comparison with FGSM, where the adversarial gaps are measured against different controls, the drop from the clean arm is normalised instead. These normalised figures are descriptive and carry no confidence interval.

3.6.5 Multiple comparisons

Chapter 4 reports well over a hundred confidence intervals, and no correction for multiple comparisons is applied. A few intervals can therefore be expected to exclude zero by chance, and conclusions are drawn from patterns that hold across budgets rather than from isolated significant cells (Section 5.2.1).

3.7 Validity and Reliability

3.7.1 Internal validity

The design aims to make every difference between arms attributable to the attacker's choice of perturbation. Several measures serve this aim.

- **A fixed victim.** The victim's parameters are frozen and its gradients disabled, and it acts deterministically. The tooling refuses to proceed if a variant's trained weights cannot be found, so no adversary can be trained or evaluated against an untrained victim.
- **Identical traffic.** The arms share the traffic seed, and the random control draws from its own generator. The records confirm the pairing: the clean arm's per-episode series is bit-identical across scopes, across domain constraints and with the first 15 episodes of the FGSM study, in every configuration (Chapter 4).
- **A matched control.** The random control uses the same admissible set and the same timing gate as the attack. Each arm gates on its own trajectory, and the realised rates of both arms are recorded; an evaluation flags any configuration in which they differ by more than five percentage points, since the arms would then differ in how often they attack and not only in the direction of the perturbation. The deterministic tie-break of Section 3.4.3 adds no randomness between them.
- **A controlled budget.** The realised attack rate is measured rather than assumed, and the tie-rationing rule was introduced after preliminary runs had shown a plain threshold failing to control it (Section 3.4.3).

- **Matched training and evaluation conditions.** Every adversary is stored with the scope and domain constraint it was trained under, the evaluation reads the scope from it and warns if the constraint differs, and every reported evaluation uses the adversary's training constraint.

3.7.2 Construct validity

Damage is measured as lost delivery relative to a random perturbation from the same set, not as the raw drop from the clean arm and not as decisions changed, which are reported separately. One gap remains between what the adversary optimises and what it is scored on: its reward is the per-hop drop fraction of Equation (3.12), while it is scored on end-to-end PDR. A significantly negative adversarial gap is therefore read as a failure of the trained adversary or of its objective, not as evidence of robustness. Similarly, the claim that the learned adversary is theoretically stronger than FGSM concerns the optimal adversary; the trained adversaries only approximate it, which is why the claim is tested empirically in the comparison with FGSM.

3.7.3 Reliability

Given a trained adversary and the traffic seed, an evaluation is designed to be reproducible: the traffic is seeded, the victim and the attack's actor act deterministically, and the random control draws from a seeded generator. The clean arms, bit-identical across separate evaluations, are consistent with this. Training is not reproducible in the same way. The adversary's weights are initialised at random, exploration noise is added to its perturbations and its updates use randomly sampled minibatches, and training was not seeded, so repeating the training of a configuration would produce a different adversary. Each configuration was trained once, and every result describes that single trained adversary. The confidence intervals of Section 3.6.2 capture the variation between episodes, not between training runs. This is the most important limitation of the study, and Section 5.2.1 discusses its consequences.

3.8 Limitations

The scope of the study was deliberately delimited in the following respects:

- Only the observation space is attacked, matching the FGSM baseline's threat model; attacks on actions, parameters, traffic or links are out of scope.
- A single topology and a single stressed traffic regime are used, without link failures, so that the results are comparable with the FGSM study and the evaluation stays within the adversaries' training conditions.

- A single perturbation radius, $\varepsilon = 0.30$, is used, and all fourteen agents are compromised. The budget of the attack is varied in time rather than in size, and partial compromise is not studied.
- Four timing budgets between 0.10 and 0.25 are evaluated, together with no budget, and every adversary is trained at $b = 0.25$ only.
- Seven of the eight combinations of the victim axes are evaluated.
- The coordination and timing mechanisms are realised with a learned adversary. Optimisation-based PGD and SAJA attacks are not implemented, and are left for future work (Section 5.3.2).

The study is also subject to limitations it could not control within its design. Each configuration was trained once and without a seed; all evaluations share one traffic seed; the comparison with FGSM rests on the 15 episodes on which the earlier study evaluated it; the reasons why the timing gate fired were recorded only for the FGSM-parity runs; and the adversary’s reward is a proxy for the delivery on which it is scored. Their consequences for the results are discussed in Section 5.2.

3.9 Summary

The research question is answered by a controlled, paired experiment in the simulator in which the victims were trained and the FGSM baseline was measured. A learned SA-MDP adversary, extended with SAJA-inspired coordination and PGD-inspired timing, is trained against each frozen victim and scored against a random control drawn from the same admissible set and gated in the same way, so the damage attributed to it is damage its learned perturbation caused. Keeping the FGSM study’s threat model, traffic and evaluation protocol makes the comparison with the baseline a like-for-like one, and varying the budget, the scope, the domain constraint and the victim separates the contribution of each mechanism and of each architectural choice. Chapter 4 presents the results, first under the FGSM-parity constraint, including the comparison with FGSM, and then under the full constraint.

STATISTICS AND RESULTS

This chapter reports what the learned observation-space adversary of Section 3.4.4 actually achieves against the victim family of Section 3.3. The question inherited from the earlier stages of this research line is narrow and comparative. The FGSM study found that perturbing every agent at every step altered a substantial fraction of routing decisions while recovering only a small part of the degradation achievable in principle. The present work asks whether that robustness survives a theoretically stronger attacker, the learned adversary, with two mechanisms. Inspired by PGD, it divides its effort across time: rather than spending a full-strength perturbation at every step, it acts on only a fraction of the steps, choosing *when* to spend a limited budget. Inspired by SAJA, it coordinates the perturbation across agents, choosing the compromised agents' perturbations together rather than one agent at a time.

Both mechanisms are evaluated. The independent scope perturbs each agent separately, so it isolates the first mechanism, dividing the attack across time. The coordinated scope adds coordination, and comparing the two scopes on identical episodes isolates what coordination contributes. Results are reported under two domain clamps, which decide which observation features are re-projected onto $[0, 1]$ after the perturbation. Section 4.2 reports both scopes under the *FGSM-parity* clamp, which reproduces the admissible set of the FGSM study, so that the attack is measured on the same terms as the baseline, and Section 4.3 compares it with the FGSM baseline under that clamp. Section 4.4 reports both scopes under the stricter *full* clamp, and Section 4.5 compares the two clamps on paired episodes.

Within each clamp and scope the results are organised around three measurements. The first is how the attack's effect varies with the timing budget, that is, with the fraction of timesteps the adversary is permitted to act on; this tests the claim of Section 3.4.3 that concentrating a limited budget on high-utilisation moments buys more damage per attacked step than attacking at every step. The second is the relationship between decisions changed and packets lost, which tests whether the flip rate reported by the earlier study is a meaningful proxy for damage at all. The third is the effect of coordination, measured as the paired difference between the coordinated and independent attacks. The FGSM-parity

runs also record why the timing gate fired, and the chapter closes with the comparison against FGSM and the effect of the domain clamp.

4.1 Reported Statistic and Experimental Protocol

Every figure and table in this chapter reports the *adversarial gap* of Equation (3.16): the paired per-episode difference in end-to-end packet delivery ratio (PDR) between the random control arm and the attack arm, expressed in percentage points. The raw drop from the clean baseline is deliberately not used as the headline number. A perturbation within the ε -ball will disturb the controller whether or not its direction was chosen adversarially, so the clean-to-attack difference conflates the damage attributable to the attacker’s choice of perturbation with the damage a random perturbation within the same ball would have caused. The adversarial gap isolates the former, and it is the only quantity that speaks to whether learning the perturbation direction was worth anything.

Three arms are run for each configuration: a clean arm with no perturbation, a random control drawing each feature’s perturbation uniformly from $[-\varepsilon, \varepsilon]$ under the same admissible set and the same timing gate, and the attack arm. The three arms are *paired*: each of the 30 evaluation episodes uses an identical traffic sequence across all arms, so the per-episode differences are matched and the reported 95% confidence intervals are computed on those paired differences. Without pairing, the between-episode variance in offered traffic would dominate the effect being measured and the intervals would be meaningless.

The victim is frozen throughout, as established in Section 3.4.1: neither actor nor critic parameters receive gradient updates during any attack or evaluation, so every number below describes an attack on a fixed controller rather than a training-time interaction. Evaluation uses $\varepsilon = 0.30$ under the L_∞ norm, 30 paired episodes of 256 timesteps in the hotspot traffic regime of Section 3.5, at an offered load factor of 2.0, with no injected link failures and a fixed traffic seed. All 14 agents are compromised in every configuration. Four sets of adversaries, one per victim variant in each set, were trained as described in Section 3.4.4: an independent and a coordinated set under each of the two domain clamps. An adversary is always evaluated under the clamp it was trained with. Each was trained for 300 episodes with the timing gate set to $b = 0.25$, the loosest budget in the sweep, so that the tighter evaluation budgets act on a subset of the state distribution seen during training.

A result is treated as evidence of a real effect only when its 95% confidence interval excludes zero; such cells are marked in the tables and drawn with filled markers in the figures. A confidence interval spanning zero is reported as a null, not as proof of robustness, since each run has a finite resolution floor that depends on the variant’s paired variance. For the tightest variants this floor is approximately 0.1 pp, meaning a true effect smaller than that would be invisible at $n = 30$ and should not be claimed either way.

4.2 Results Under the FGSM-Parity Domain Clamp

Every result in this section was obtained under the *FGSM-parity domain clamp*: after the ℓ_∞ projection, only the first four features of each agent’s observation, four of its six available-bandwidth features, are re-projected onto $[0, 1]$, and every other feature may move anywhere within the ε -ball. This is the domain constraint $\mathcal{D}$ of Equation (3.3), the one used by the FGSM study of Section 3.2.2, so the attack and its random control draw from exactly the admissible set within which the baseline perturbed. Separate sets of seven independent-scope and seven coordinated-scope adversaries were trained for this clamp, and every one of the 70 evaluation records stores the clamp it was run under. The clean arm depends on neither the adversary nor the clamp, and its per-episode delivery series is bit-identical to that of the full-clamp evaluation in all 70 configurations.

This clamp is the looser of the two: every perturbation admissible under the full clamp of Section 4.4 is also admissible here. That does not make the damage measured here an upper bound on the damage measured there. Each clamp has its own trained adversary, and a learned adversary is not guaranteed to exploit a larger set at least as well as a smaller one; Section 4.5 shows that on several variants it does not.

4.2.1 Independent scope: sensitivity to the timing budget

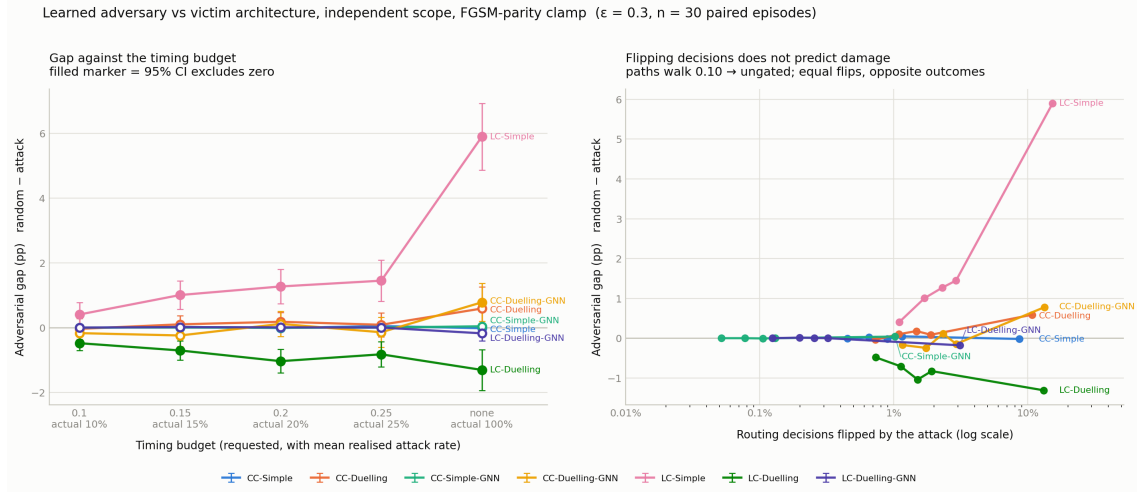


Figure 4.1: Independent scope, FGSM-parity domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes. **Left:** gap against the requested timing budget, with the mean realised attack rate printed beneath each tick and the ungated run included as the 100% reference; filled markers denote a 95% confidence interval excluding zero, hollow markers a null. **Right:** the same gaps plotted against the fraction of routing decisions the attack actually changed, on a logarithmic axis, with each variant drawn as one path walking from $b = 0.10$ through to the ungated run. A thin line joins a label to its series where the label had to be moved clear of the data.

The timing gate spends the budget it is given: the four requested budgets realise, averaged over the seven variants, as 10.0%, 15.0%, 19.9% and 24.9% of steps. Figure 4.1 and

Table 4.1 show a different architectural pattern from the one found under the full clamp. A single variant is clearly vulnerable. LC-Simple is attacked significantly at every budget, its gap rising from +0.40 pp at $b = 0.10$ to +1.45 pp at $b = 0.25$ and +5.90 pp ungated, the largest gap measured in this study. Ungated, its delivery ratio falls to 71.8%, against 77.3% clean and 77.7% under the random control. CC-Duelling-GNN separates from zero only ungated, at +0.78 pp, and CC-Duelling does not separate at all, reaching +0.59 pp ungated with an interval that spans zero. CC-Simple, CC-Simple-GNN and LC-Duelling-GNN stay within 0.2 pp of zero at every budget.

Table 4.1: Independent scope, FGSM-parity domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero. Positive values indicate the attack cost the victim more than the random control drawn from the same ε -ball; negative values indicate the reverse.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	-0.01	+0.02	-0.01	+0.05	-0.02
CC-Duelling	-0.04	+0.10	+0.18	+0.09	+0.59
CC-Simple-GNN	+0.00	+0.00	-0.01	+0.00	+0.04
CC-Duelling-GNN	-0.17	-0.24	+0.11	-0.15	+0.78
LC-Simple	+0.40	+1.01	+1.27	+1.45	+5.90
LC-Duelling	-0.48	-0.71	-1.04	-0.83	-1.31
LC-Duelling-GNN	+0.00	+0.01	+0.01	+0.00	-0.18

LC-Duelling is an exception in the other direction. Its gap is significantly *negative* at every budget, from -0.48 pp at $b = 0.10$ to -1.31 pp ungated. Ungated, the victim delivers 78.6% of packets under attack, against 77.3% under the random control and 76.9% with no perturbation at all, so the adversary trained against this victim steers it to deliver more than it would unperturbed. This is not evidence of robustness. It shows that the direction the adversary learned is systematically wrong for this victim. The adversary maximises the per-step drop fraction of Equation (3.12) rather than the end-to-end delivery ratio on which it is scored, and either the two objectives come apart on this victim or training converged to a poor direction; these runs do not separate the two.

In the independent scope under this clamp, concentrating the budget on critical states does not buy more damage per attacked step. Normalising LC-Simple’s gap by the realised attack rate gives 4.0 pp per unit rate at $b = 0.10$, 6.7 at $b = 0.15$, 6.4 at $b = 0.20$ and 5.9 at $b = 0.25$, against 5.9 ungated. Beyond the tightest budget its damage grows in proportion to the number of steps attacked, and the tightest budget is the least efficient rather than the most. No other variant has a significant positive gap at any gated budget, so none shows an efficiency gain either. The efficiency found under the full clamp in Section 4.4.1 is therefore a property of the adversaries trained there, not of dividing the attack across time as such. As under the full clamp, the adversaries were trained with the gate at $b = 0.25$, so the ungated column asks them to act on states they did not encounter during training. For LC-Simple this costs nothing: its ungated gap is four times its gap at $b = 0.25$, for four

times the number of steps attacked.

4.2.2 Independent scope: decision flips versus delivered damage

The right panel of Figure 4.1 and Table 4.2 show the flip rate failing as a measure of damage more starkly than under the full clamp. Ungated, LC-Simple, CC-Duelling-GNN and LC-Duelling change 15.3%, 13.3% and 13.2% of routing decisions, within about two percentage points of one another, yet their gaps are +5.90, +0.78 and -1.31 pp, a spread of more than seven percentage points. The same holds at a gated budget: at $b = 0.25$, LC-Simple and CC-Duelling-GNN both change 2.92% of decisions, with gaps of +1.45 and -0.15 pp. The random control again changes many decisions on its own, between 9.6% and 10.7% on the four variants with the highest flip rates.

Table 4.2: Independent scope, FGSM-parity domain clamp. Ungated results at $\varepsilon = 0.30$, $n = 30$ paired episodes: adversarial gap with its 95% confidence interval, and the fraction of routing decisions changed by the attack and by the random control.

Variant	Gap (pp)	95% CI	Attack flips	Random flips
CC-Simple	-0.02	$[-0.17, +0.14]$	8.68%	5.23%
CC-Duelling	$+0.59$	$[-0.08, +1.26]$	10.77%	9.62%
CC-Simple-GNN	$+0.04$	$[-0.02, +0.10]$	1.02%	0.27%
CC-Duelling-GNN	$+0.78$	$[+0.20, +1.36]$	13.34%	10.63%
LC-Simple	$+5.90$	$[+4.87, +6.93]$	15.32%	10.74%
LC-Duelling	-1.31	$[-1.94, -0.68]$	13.15%	9.87%
LC-Duelling-GNN	-0.18	$[-0.42, +0.06]$	3.11%	1.11%

The two GNN variants that resist the attack show the same two behaviours as under the full clamp (Section 4.4.2). CC-Simple-GNN changes only 1.02% of decisions ungated, and at most 0.13% at any gated budget, so the perturbation rarely moves the selected path at all. LC-Duelling-GNN changes 3.11% ungated and still returns a gap of -0.18 pp: its routing does change, and the network absorbs the change.

4.2.3 Independent scope: what triggered the attacks

The FGSM-parity runs are the first in which the timing gate recorded why it acted. For every attacked step it logs which case of Equation (3.10) admitted the step, a score strictly above the threshold or a tie at the threshold admitted by rationing, together with the congestion band the score fell in: saturated ($c_t \geq 1.00$), near saturation ($0.90 \leq c_t < 1.00$), high congestion ($0.75 \leq c_t < 0.90$) or moderate ($c_t < 0.75$). Figure A.1, in Appendix A, shows each trigger’s share of the attack arm’s attacked steps. The random control passes through the same gate, and its composition is nearly identical: at $b = 0.15$, for instance, its tie share differs from the attack arm’s by at most three percentage points for every variant.

Three findings stand out. First, no attacked step in any run, in either arm, scored 1.00 or more, or less than 0.75. The point mass in the score distribution that the rationing rule of Section 3.4.3 exists to handle lies at 0.90 rather than at full saturation. At $b = 0.10$

and $b = 0.15$ the mean threshold at which the gate fired is 0.90 for every variant, and the tied steps are the steps sitting on that point mass. Its stored value falls a floating-point rounding error below 0.90, which is why ties are recorded in the high-congestion band although they sit at that band's upper edge.

Second, the most congested states always fire. Steps scoring between 0.90 and 1.00 are admitted at every budget, in a nearly constant number per variant: 321 for CC-Simple and between 505 and 599 for the others. Their share of the attacks therefore falls from 42–77% at $b = 0.10$ to 17–32% at $b = 0.25$.

Third, the route by which the rest of the budget is spent changes with the budget. At $b = 0.10$ and $b = 0.15$ the threshold sits on the point mass, and ties admitted by rationing account for 23–58% and 45–66% of the attacks respectively. On the observed score sequences, admitting every tie would have raised the attack rate to between 17% and 21% at both budgets, collapsing the two tightest budgets into nearly the same experiment; rationing is what keeps them apart. From $b = 0.20$ the threshold begins to fall below the point mass, and by $b = 0.25$ it lies between 0.82 and 0.90. The steps on the point mass then pass as strictly above the threshold, and above-threshold steps in the high-congestion band, the point mass among them, make up 57–77% of the attacks at $b = 0.25$, while ties fall to 6–11%.

4.2.4 Coordinated scope

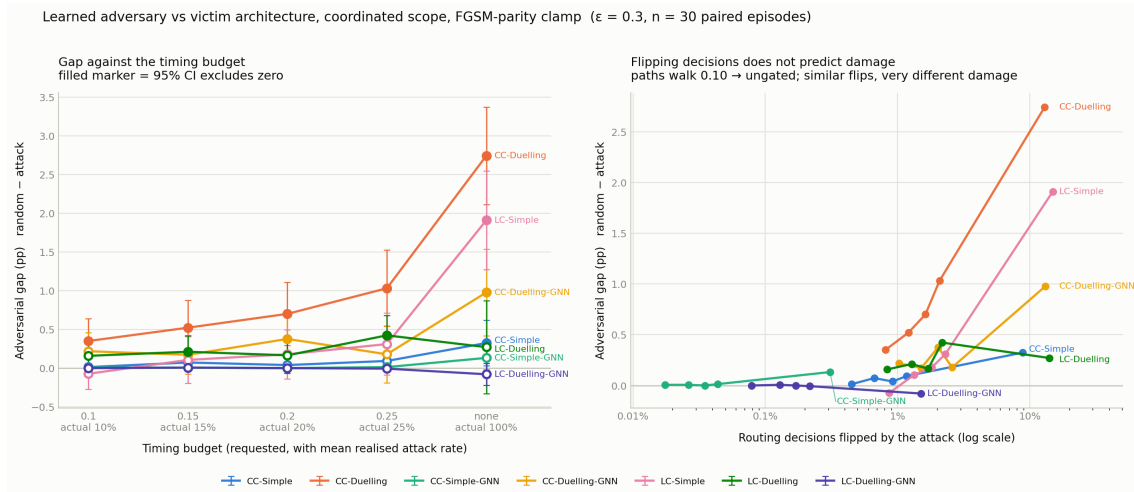


Figure 4.2: Coordinated scope, FGSM-parity domain clamp. Adversarial gap for all seven victim variants at $\epsilon = 0.30$, $n = 30$ paired episodes, with the perturbation emitted jointly for the whole compromised group. Panels and markers as in Figure 4.1. A thin line joins a label to its series where the label had to be moved clear of the data.

Figure 4.2 and Table 4.3 repeat the sweep with the perturbation coordinated across the compromised agents. The timing gate again spends the budget it is given, realising 10.1%, 15.0%, 19.9% and 24.9% of steps on average at the four budgets. The architectural pattern differs from that of the independent scope under the same clamp. CC-Duelling becomes

the most vulnerable variant and is attacked significantly at every budget, its gap rising from +0.35 pp at $b = 0.10$ to +1.03 pp at $b = 0.25$ and +2.74 pp ungated. CC-Duelling-GNN separates from zero at $b = 0.20$ (+0.38 pp) and ungated (+0.98 pp), and CC-Simple ungated only (+0.32 pp). LC-Simple, the variant most damaged by the independent attack, is harmed significantly only when every step is attacked (+1.91 pp). LC-Duelling, which the independent adversary helped, is now harmed, significantly at $b = 0.15$ (+0.21 pp) and $b = 0.25$ (+0.42 pp), although not ungated. CC-Simple-GNN and LC-Duelling-GNN stay within 0.15 pp of zero throughout.

Table 4.3: Coordinated scope, FGSM-parity domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	+0.01	+0.07	+0.04	+0.09	+0.32
CC-Duelling	+0.35	+0.52	+0.70	+1.03	+2.74
CC-Simple-GNN	+0.01	+0.01	+0.00	+0.01	+0.13
CC-Duelling-GNN	+0.22	+0.17	+0.38	+0.18	+0.98
LC-Simple	-0.07	+0.11	+0.18	+0.31	+1.91
LC-Duelling	+0.16	+0.21	+0.17	+0.42	+0.27
LC-Duelling-GNN	+0.00	+0.01	+0.00	-0.01	-0.08

In this scope, concentrating the budget on critical states does buy more damage per attacked step for the most vulnerable variant. CC-Duelling’s gap per unit of realised attack rate is 3.5 pp at $b = 0.10$ and 4.2 pp at $b = 0.25$, against 2.7 pp ungated. The decisions changed again fail to predict the damage: ungated, CC-Duelling, CC-Duelling-GNN, LC-Duelling and LC-Simple change between 13.0% and 14.9% of routing decisions, yet their gaps range from +0.27 to +2.74 pp.

The timing gate behaves as it does in the independent scope (Figure A.2, in Appendix A). No attacked step scored 1.00 or more, or less than 0.75. Steps scoring between 0.90 and 1.00 fire in a nearly constant number per variant at every budget, so their share of the attacks falls from 42–78% at $b = 0.10$ to 17–32% at $b = 0.25$. Ties admitted by rationing make up 22–58% of the attacks at $b = 0.10$ and 44–66% at $b = 0.15$; admitting every tie would have raised the attack rate to between 17.5% and 21.3% at both budgets. At $b = 0.25$ the threshold lies between 0.82 and 0.90, above-threshold steps in the high-congestion band make up 58–77% of the attacks, and ties only 3–11%. The gate’s score depends only on link utilisation, and the close match with Figure A.1 indicates that the composition of the triggers is set by the network and the budget rather than by how the perturbation is chosen.

4.2.5 Effect of coordination

Under this clamp too, the clean arm and the random control are bit-identical between the two scopes in all 35 configurations, so the effect of coordination is the exact paired

difference between the two attack arms of Equation (3.17) (Section 3.6.3). Table 4.4 reports it.

Table 4.4: Effect of coordination under the FGSM-parity domain clamp: coordinated minus independent adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean coordination increased the damage. Bold entries have a 95% confidence interval on the paired difference excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	+0.02	+0.05	+0.05	+0.05	+0.34
CC-Duelling	+0.39	+0.42	+0.52	+0.95	+2.16
CC-Simple-GNN	+0.01	+0.01	+0.01	+0.01	+0.09
CC-Duelling-GNN	+0.39	+0.42	+0.26	+0.32	+0.20
LC-Simple	-0.48	-0.90	-1.09	-1.14	-3.99
LC-Duelling	+0.64	+0.92	+1.20	+1.25	+1.58
LC-Duelling-GNN	+0.00	-0.01	-0.01	-0.01	+0.10

Coordination changes the result significantly on four variants, in both directions. It increases the damage done to LC-Duelling at every budget, by 0.64 to 1.58 pp, turning the variant the independent adversary helped into one the attack harms. It increases the damage to CC-Duelling from $b = 0.15$ onwards, by 0.42 to 2.16 pp, and to CC-Duelling-GNN at the two tightest budgets, by 0.39 and 0.42 pp. It reduces the damage done to LC-Simple at every budget, by 0.48 to 3.99 pp. CC-Simple gains 0.34 pp only when every step is attacked, and the two remaining GNN variants are unaffected.

These effects do not match those found under the full clamp (Section 4.4.4), where coordination made LC-Simple attackable and reduced the damage done to CC-Duelling-GNN. Here coordination hurts LC-Simple, helps CC-Duelling-GNN, and its largest gains fall on one central-critic and one local-critic variant, which the local-critic explanation offered in Section 4.4.4 does not predict. What coordination contributes therefore depends on the trained adversaries being compared, not only on the victim’s critic domain.

4.3 Comparison with the FGSM Baseline

This section compares the learned adversary with the FGSM baseline of Section 3.2.2, using only the FGSM-parity results of Section 4.2, so that both attacks perturb the same frozen agents within the same admissible set and the same $\varepsilon = 0.30$. The FGSM baseline was evaluated on 15 episodes at the same load and traffic seed, with no link failures, and its clean arm is bit-identical to the first 15 episodes of every FGSM-parity evaluation in all 70 configurations. The comparison is therefore paired on those 15 episodes. FGSM applies a single-step perturbation to every agent at every time step.

The two studies used different random controls: FGSM’s draws a random sign of size ε for each feature, while the learned adversary’s draws each feature uniformly from the ε -ball. Their adversarial gaps are measured against different references and are not

differentiated here. Because the clean arms are identical, the attack arms themselves ran on the same traffic against the same victim, and they are compared directly by Δ^{FGSM} of Equation (3.19), with a paired 95% confidence interval using $t_{0.975,14}$. A positive value means the learned adversary did more damage than FGSM. With only 15 pairs, the intervals are wider than those of the preceding sections.

For reference, Table 4.5 gives FGSM’s own adversarial gap on these episodes, against its own random control. It separates from zero on three variants: CC-Duelling (+2.53 pp), LC-Simple (+3.27 pp) and LC-Duelling (+1.24 pp).

Table 4.5: FGSM baseline under the FGSM-parity domain clamp, $\varepsilon = 0.30$, no link failures, $n = 15$ paired episodes: adversarial gap against FGSM’s own random control, with its 95% confidence interval, and the fraction of routing decisions the attack changed. Bold entries have a confidence interval excluding zero.

Variant	Gap (pp)	95% CI	Attack flips
CC-Simple	+0.37	[−0.33, +1.08]	25.73%
CC-Duelling	+2.53	[+0.99, +4.07]	20.05%
CC-Simple-GNN	−0.01	[−0.49, +0.46]	1.41%
CC-Duelling-GNN	+1.01	[−0.09, +2.10]	12.58%
LC-Simple	+3.27	[+1.82, +4.71]	19.38%
LC-Duelling	+1.24	[+0.54, +1.95]	18.36%
LC-Duelling-GNN	−0.15	[−0.30, +0.01]	2.57%

4.3.1 Independent scope with timing budget

Table 4.6: Independent learned adversary against FGSM, FGSM-parity domain clamp: Δ^{FGSM} of Equation (3.19), in percentage points, by victim variant and the learned adversary’s requested timing budget ($\varepsilon = 0.30$, $n = 15$ paired episodes). FGSM attacks every step in every column. Positive values mean the learned adversary did more damage. Bold entries have a 95% confidence interval excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	−0.37	−0.36	−0.36	−0.38	−0.41
CC-Duelling	−2.29	−2.28	−2.21	−2.35	−2.22
CC-Simple-GNN	+0.05	+0.05	+0.05	+0.05	+0.03
CC-Duelling-GNN	−1.77	−1.44	−1.47	−1.53	−0.38
LC-Simple	−2.63	−2.18	−1.72	−1.92	+2.21
LC-Duelling	−0.83	−1.12	−1.38	−1.48	−1.85
LC-Duelling-GNN	+0.13	+0.13	+0.13	+0.12	+0.03

Table 4.6 shows that the independent learned adversary does not do more damage than FGSM at any timing budget. On four variants it does significantly less at every budget: CC-Duelling by 2.21 to 2.35 pp, LC-Simple by 1.72 to 2.63 pp, CC-Duelling-GNN by 1.44 to 1.77 pp and LC-Duelling by 0.83 to 1.48 pp. Part of this shortfall is by construction, since the gated attack acts on only 10 to 25% of the steps while FGSM acts on all of them. Normalised by the realised attack rate, the raw drop in delivery that the gated attack

causes on LC-Simple is about 4.3 to 5.6 pp per unit rate at $b = 0.15$ and $b = 0.20$, against 2.8 pp for FGSM, so on that variant the learned attack does more damage per attacked step even where its total damage is lower. This normalisation is descriptive and carries no confidence interval.

When both attacks act on every step, the ungated column compares them on equal terms. The learned adversary then does 2.21 pp more damage than FGSM on LC-Simple, the only configuration in which it beats the baseline, but still 2.22 pp less on CC-Duelling and 1.85 pp less on LC-Duelling, the variant on which it improves delivery (Section 4.2.1). On CC-Simple, CC-Simple-GNN and LC-Duelling-GNN neither attack does measurable damage, and the two do not differ at any budget.

4.3.2 Coordinated scope

Table 4.7: Coordinated learned adversary against FGSM, FGSM-parity domain clamp: Δ^{FGSM} of Equation (3.19), in percentage points, by victim variant and the learned adversary’s requested timing budget ($\varepsilon = 0.30$, $n = 15$ paired episodes). Positive values mean the learned adversary did more damage. Bold entries have a 95% confidence interval excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	-0.38	-0.30	-0.29	-0.29	-0.15
CC-Duelling	-1.89	-1.79	-1.53	-1.19	+0.52
CC-Simple-GNN	+0.08	+0.08	+0.08	+0.09	+0.17
CC-Duelling-GNN	-1.46	-1.36	-1.42	-1.40	-0.56
LC-Simple	-2.83	-2.75	-2.35	-2.51	-1.40
LC-Duelling	-0.30	-0.20	-0.12	-0.17	-0.37
LC-Duelling-GNN	+0.13	+0.13	+0.13	+0.12	+0.20

Adding coordination brings the learned adversary closer to FGSM on the duelling variants, but not on LC-Simple (Table 4.7). On CC-Duelling it still does significantly less damage than FGSM at the two tightest budgets, by 1.89 and 1.79 pp, but the difference is no longer significant from $b = 0.20$, and ungated the coordinated attack does 0.52 pp more than FGSM, within the interval. Per attacked step it exceeds FGSM on this variant at $b = 0.25$, with a raw drop of about 3.3 pp per unit rate against 2.0 pp. On LC-Duelling, where the independent attack fell significantly short of FGSM at every budget, the coordinated attack cannot be distinguished from FGSM at any budget. CC-Duelling-GNN remains significantly below FGSM at three of the four gated budgets, by 1.40 to 1.46 pp. LC-Simple moves the other way: the coordinated attack does significantly less damage than FGSM at every gated budget, by 2.35 to 2.83 pp, and ungated it no longer beats the baseline, falling 1.40 pp short within the interval.

Taken together, when both attacks are confined to the FGSM study’s admissible set, the learned adversary does not do significantly more damage than FGSM in any configuration except one, the ungated independent attack on LC-Simple. In every other configuration it matches FGSM or falls short of it, including on the three variants against which FGSM

itself separates from its random control. Apart from that one case, its advantage appears only per attacked step, on individual variants, and does not carry through to more total damage than the single-step baseline.

This outcome bears directly on the robustness of the controller. The learned adversary is theoretically the stronger attacker (Section 3.4.4): it searches over perturbation policies that include FGSM’s, and it can additionally coordinate across agents and choose when to act. That such an attacker, trained against the victim, ends up doing about as much damage as a single gradient step supports the reading that the controller is robust to observation-space attacks of this size, rather than the reading that the FGSM baseline was simply too weak to find its vulnerabilities. As with every result in this chapter, the conclusion concerns the adversaries that were trained, not a bound on the damage an optimal attack could do.

4.4 Results Under the Full Domain Clamp

Every result in this section, for both scopes and for both the attack and the random control, was obtained under the *full domain clamp*: after the ℓ_∞ projection, every feature of the perturbed observation is re-projected onto $[0, 1]$. The clamp is recorded in each evaluation record, and it is the same for all 70 configurations reported here. As set out in Section 3.4.4, this is stricter than the domain constraint of the FGSM study of Section 3.2.2, which clamped only the first four features of each observation. The admissible set used here is therefore a strict subset of the one used by the baseline, with roughly a third of the available perturbation range removed.

Three consequences follow. First, because the attack and its random control share the same set, the adversarial gap remains internally valid, and every comparison *within* this section is sound, including the comparison between scopes. Second, the damage measured here is not a lower bound on the damage attained under the baseline’s admissible set, even though that set is larger: the adversaries of Section 4.2 were trained separately, and Section 4.5 shows them doing less damage than the adversaries of this section on several variants. Third, a direct comparison of absolute PDR figures against the published FGSM numbers is not licensed, since the attacker here operates under a different constraint; that comparison belongs to the FGSM-parity clamp.

4.4.1 Independent scope: sensitivity to the timing budget

The left panel of Figure 4.3 shows the gap rising monotonically with the timing budget for every variant that is attackable at all. The realised attack rates printed under the ticks confirm that the gate spends the budget it is given: requested budgets of 0.10, 0.15, 0.20 and 0.25 realise, averaged over the seven variants, as 10.0%, 15.0%, 19.9% and 24.9% respectively. This matters because the critical-state score has a point mass, which Section 4.2.3 locates at 0.90, so a large block of timesteps ties at exactly the trigger threshold

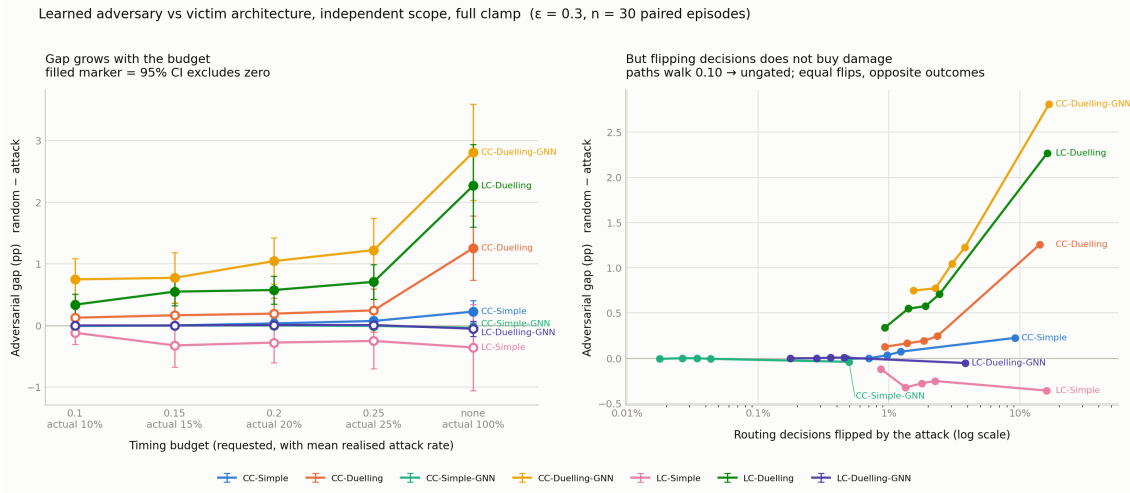


Figure 4.3: Independent scope, full domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes. **Left:** gap against the requested timing budget, with the mean realised attack rate printed beneath each tick and the ungated run included as the 100% reference; filled markers denote a 95% confidence interval excluding zero, hollow markers a null. **Right:** the same gaps plotted against the fraction of routing decisions the attack actually changed, on a logarithmic axis, with each variant drawn as one path walking from $b = 0.10$ through to the ungated run.

at the tightest budgets; without the tie rationing described in Section 3.4.3 the realised rate cannot fall below the fraction of steps on that point mass, and the two tightest budgets collapse into nearly the same experiment.

Table 4.8 gives the underlying values. The architectural pattern is sharp: the duelling value head is the vulnerable axis. All three duelling variants without a local-critic GNN reach a significant gap at full rate, led by CC-Duelling-GNN at +2.81 pp and LC-Duelling at +2.27 pp, while every simple-head variant remains at or below +0.22 pp. Three variants, CC-Simple-GNN, LC-Simple and LC-Duelling-GNN, never separate from zero at any budget.

Table 4.8: Independent scope, full domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero. Positive values indicate the attack cost the victim more than the random control drawn from the same ε -ball; negative values indicate the reverse.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	-0.01	+0.00	+0.03	+0.07	+0.22
CC-Duelling	+0.13	+0.17	+0.19	+0.24	+1.26
CC-Simple-GNN	-0.01	+0.00	+0.00	-0.01	-0.04
CC-Duelling-GNN	+0.75	+0.77	+1.04	+1.22	+2.81
LC-Simple	-0.12	-0.32	-0.28	-0.25	-0.36
LC-Duelling	+0.34	+0.55	+0.58	+0.71	+2.27
LC-Duelling-GNN	+0.00	+0.00	+0.01	+0.01	-0.05

Under this clamp, the budget sweep also supports the efficiency claim that motivated

the timing gate. Normalising each gap by the realised attack rate gives the damage bought per attacked timestep, and for CC-Duelling-GNN this falls from 7.3 pp per unit rate at $b = 0.10$ to 2.8 pp when ungated, a factor of 2.6; for LC-Duelling the same ratio is 1.5. For these adversaries, concentrating the budget on high-utilisation states is therefore not merely cheaper but more efficient per step spent, which is the property that motivates dividing the attack across time in the manner of PGD. The effect is not universal: CC-Duelling shows essentially no efficiency gain, its damage scaling in proportion to the number of steps attacked; under the FGSM-parity clamp the independent attack shows no efficiency gain on any variant (Section 4.2.1), while the coordinated attack shows one on CC-Duelling (Section 4.2.4).

One caveat applies to the ungated column. Because the adversaries were trained with the gate at $b = 0.25$, evaluating them without a gate asks them to act on the remaining 75% of states they never encountered during training. That the ungated run is nonetheless the strongest point for every attackable variant indicates the learned perturbation direction generalises beyond the state distribution it was fitted on, rather than being specialised to high-utilisation moments.

4.4.2 Independent scope: decision flips versus delivered damage

The right panel of Figure 4.3 addresses the assumption implicit in reporting flip rates as an attack outcome. Within any single variant the relationship holds in the expected direction: as the budget loosens, more routing decisions change and more damage follows, so each path slopes upward. Across variants the relationship fails completely. At the right-hand edge of the panel four variants sit within a narrow band of 14–17% of decisions flipped, yet their gaps span from -0.36 pp to $+2.81$ pp, a spread of more than three percentage points at essentially identical flip rates. Table 4.9 gives these figures directly.

Table 4.9: Independent scope, full domain clamp. Ungated results at $\varepsilon = 0.30$, $n = 30$ paired episodes: adversarial gap with its 95% confidence interval, and the fraction of routing decisions changed by the attack and by the random control.

Variant	Gap (pp)	95% CI	Attack flips	Random flips
CC-Simple	+0.22	[+0.04, +0.41]	9.15%	6.80%
CC-Duelling	+1.26	[+0.73, +1.78]	14.14%	12.02%
CC-Simple-GNN	-0.04	[-0.10, +0.02]	0.50%	0.34%
CC-Duelling-GNN	+2.81	[+2.03, +3.59]	16.69%	15.45%
LC-Simple	-0.36	[-1.06, +0.34]	15.94%	12.91%
LC-Duelling	+2.27	[+1.60, +2.94]	16.16%	11.98%
LC-Duelling-GNN	-0.05	[-0.18, +0.07]	3.82%	2.18%

LC-Simple is the clearest counterexample. Its path in the right panel runs downward as it moves right, so the more routing decisions the attack changes, the *better* the network performs relative to the random control. Flipping a routing decision is evidently not the same as flipping it to a worse path, and on this variant the learned direction is, on balance,

no worse than noise. The random control is itself instructive here: it changes 12–15% of decisions on the high-flip variants, which means a raw flip count says almost nothing about whether an attacker is doing anything an untargeted disturbance would not.

The logarithmic axis separates two failure modes that a linear axis renders identical. CC-Simple-GNN barely moves the argmax, flipping between 0.02% and 0.50% of decisions across the entire sweep; the perturbation is not being absorbed downstream, it is mostly failing to change the selected path in the first place. This is the signature either of a policy whose decision boundaries lie beyond the ε -ball, or of an adversary that has failed to learn a perturbation able to cross them. Since the learned adversary never differentiates the victim, gradient masking in the usual sense cannot be the cause. LC-Duelling-GNN behaves quite differently: it flips 3.82% of decisions at full rate, more than seven times CC-Simple-GNN’s rate, and still returns a gap of -0.05 pp. Here the routing genuinely changes and the packets arrive regardless, which is absorption in the network rather than insensitivity in the policy. The distinction matters for what would follow: the first case might yield to a stronger optimiser or a larger budget, whereas the second reflects path redundancy that no observation-space perturbation of this form is likely to overcome.

4.4.3 Coordinated scope

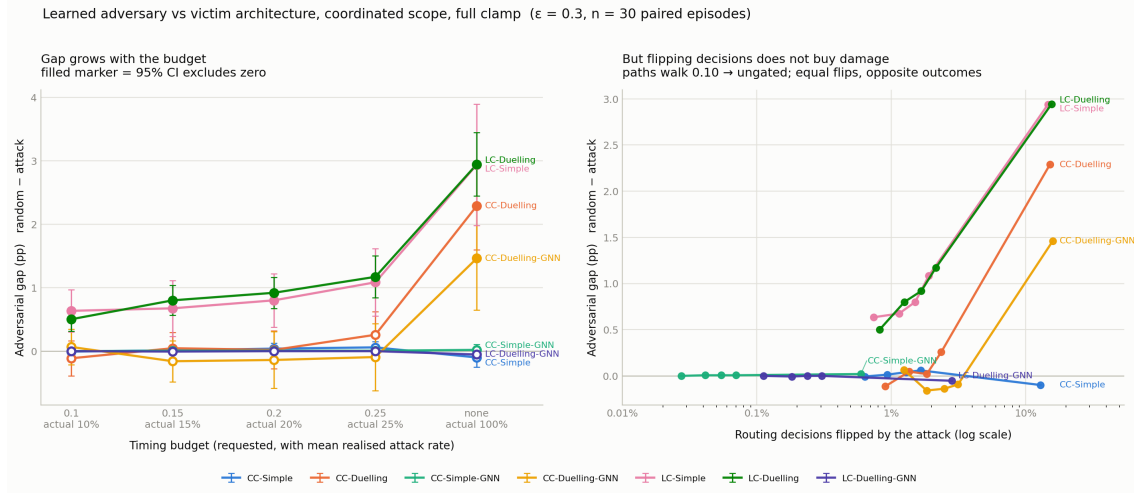


Figure 4.4: Coordinated scope, full domain clamp. Adversarial gap for all seven victim variants at $\varepsilon = 0.30$, $n = 30$ paired episodes, with the perturbation emitted jointly for the whole compromised group. Panels and markers as in Figure 4.3. A thin line joins a label to its series where the label had to be moved clear of the data.

Figure 4.4 repeats the sweep with the perturbation coordinated across the compromised agents, and Table 4.10 gives the values. The timing gate behaves identically in both scopes, realising 10.0%, 15.0%, 19.9% and 24.9% of steps at the four budgets, so any difference from the independent scope is due to the coordination of the perturbation rather than to how often it is applied.

The architectural pattern, however, changes markedly. The two local-critic variants without GNN encoding become the most vulnerable, and both are attacked significantly at *every* budget. LC-Simple, which the independent full-clamp attack never harmed, rises monotonically from +0.63 pp at $b = 0.10$ to +1.08 pp at $b = 0.25$ and +2.94 pp ungated. LC-Duelling follows the same course, from +0.50 pp to +1.17 pp and +2.94 pp. The central-critic duelling variants, in contrast, are attacked significantly only when the adversary acts on every step: CC-Duelling reaches +2.29 pp and CC-Duelling-GNN +1.46 pp ungated, but neither separates from zero at any gated budget. CC-Simple, CC-Simple-GNN and LC-Duelling-GNN remain unaffected throughout.

Table 4.10: Coordinated scope, full domain clamp. Adversarial gap in percentage points by victim variant and requested timing budget, at $\varepsilon = 0.30$ and $n = 30$ paired episodes. Bold entries have a 95% confidence interval excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	-0.01	+0.01	+0.04	+0.06	-0.10
CC-Duelling	-0.11	+0.05	+0.02	+0.26	+2.29
CC-Simple-GNN	+0.00	+0.01	+0.01	+0.01	+0.02
CC-Duelling-GNN	+0.07	-0.16	-0.14	-0.09	+1.46
LC-Simple	+0.63	+0.67	+0.80	+1.08	+2.94
LC-Duelling	+0.50	+0.80	+0.92	+1.17	+2.94
LC-Duelling-GNN	+0.00	-0.01	+0.00	+0.00	-0.05

Where coordination works, dividing the attack across time remains efficient. The damage bought per attacked step for LC-Simple is 6.3 pp per unit rate at $b = 0.10$ against 2.9 pp ungated, a factor of 2.2, and for LC-Duelling it is 5.0 pp against 2.9 pp, a factor of 1.7. The right panel repeats the lesson of Section 4.4.2: the paths of LC-Simple and LC-Duelling nearly coincide, ending at +2.94 pp with 14.7% and 15.6% of decisions flipped, while CC-Duelling-GNN ends at only +1.46 pp despite flipping 16.0%, more than either.

4.4.4 Effect of coordination

The two scopes can be compared exactly. Both were evaluated on the same traffic seed, and because neither the clean arm nor the random control depends on the adversary, their per-episode PDR series are bit-identical across the two scopes in all 35 configurations. The difference between the coordinated and independent adversarial gaps is then exactly the per-episode difference between the two attack arms, Δ^{coord} of Equation (3.17), which is paired within each episode and carries a confidence interval of the same form as Equation (3.16). A positive value means coordination increased the damage done. Table 4.11 reports it for every configuration.

Coordination does something, but not uniformly. On the two local-critic variants without GNN encoding it increases damage significantly at every budget. For LC-Simple the gain grows from +0.75 pp at $b = 0.10$ to +3.29 pp ungated, turning a variant the

Table 4.11: Effect of coordination under the full domain clamp: coordinated minus independent adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean coordination increased the damage. Bold entries have a 95% confidence interval on the paired difference of Equation (3.17) excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	+0.00	+0.01	+0.01	-0.01	-0.32
CC-Duelling	-0.24	-0.12	-0.17	+0.01	+1.03
CC-Simple-GNN	+0.01	+0.01	+0.01	+0.01	+0.06
CC-Duelling-GNN	-0.68	-0.93	-1.18	-1.32	-1.35
LC-Simple	+0.75	+1.00	+1.08	+1.34	+3.29
LC-Duelling	+0.17	+0.25	+0.34	+0.46	+0.67
LC-Duelling-GNN	+0.00	-0.01	-0.01	-0.01	+0.00

independent full-clamp attack could not harm into one of the most damaged in the study. For LC-Duelling, already vulnerable, the gain grows from +0.17 pp to +0.67 pp. On the central-critic variants the effect is mixed. It is nil at every gated budget except for CC-Duelling-GNN, where coordination significantly *reduces* damage at every budget, by between 0.68 and 1.35 pp. It appears only when attacking every step, where it helps CC-Duelling (+1.03 pp) and hurts CC-Simple (-0.32 pp). The two GNN variants that resisted the independent attack are unaffected by coordination as well.

The gains do not come from changing more decisions. On LC-Simple at $b = 0.25$ the coordinated attack flips fewer routing decisions than the independent one, 1.91% against 2.24%, while the gap moves from -0.25 pp to +1.08 pp; ungated, its flip rate falls from 15.94% to 14.65% as the gap rises by more than three percentage points. The converse also occurs: on CC-Simple coordination raises the ungated flip rate from 9.15% to 12.94% while the gap falls. What coordination changes is *which* decisions change together, and only that relates to delivered damage.

The concentration of the gains on the local-critic variants is consistent with how the two critic domains of Equation (3.1) are trained. A local critic conditions only on its own agent’s observation and action, so the policies it trains never receive a signal about how several agents’ routing choices combine on a shared link. A central critic conditions on the joint action, so its policies are trained against a value model that accounts for exactly those joint consequences. A perturbation that steers several agents onto the same link targets what the local critic cannot see. This explanation is consistent with the pattern but is not established by it: in particular, it does not account for why the effect disappears for LC-Duelling-GNN, and it does not predict the loss on CC-Duelling-GNN.

That loss calls for a specific caution. As described in Section 3.4.4, the coordinated adversary acts on the joint observation of all fourteen compromised agents, yet stores one replay transition per step rather than one per agent, so it learns from far fewer and far higher-dimensional samples than the independent adversary does. The large gains on the local-critic variants, obtained under the same condition, show that the reduced sample

count alone does not explain the overall pattern. It cannot, however, be ruled out as the cause of the deficit on CC-Duelling-GNN in particular, and separating an under-trained adversary from a genuine effect of coordination on that variant would require an analysis of the coordinated adversary’s learning signal.

4.5 Effect of the Domain Clamp

The two clamps can be compared on paired episodes in both scopes. The clean arm is bit-identical across the clamps, as noted in Section 4.2, but the random control is not, since each clamp’s control draws from its own admissible set. The difference between the two adversarial gaps, Δ^{clamp} of Equation (3.18), is nevertheless paired within each episode and carries a confidence interval of the same form as Equation (3.16). A positive value means the parity-clamp adversary did more damage, relative to its own random control, than the full-clamp adversary. Tables 4.12 and 4.13 report it for every configuration of the independent and coordinated scopes.

Table 4.12: Effect of the domain clamp in the independent scope: FGSM-parity minus full clamp adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean the parity-clamp adversary did more damage. Bold entries have a 95% confidence interval on the paired difference of Equation (3.18) excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	+0.00	+0.02	−0.05	−0.03	−0.24
CC-Duelling	−0.17	−0.07	−0.01	−0.16	−0.67
CC-Simple-GNN	+0.01	+0.00	−0.01	+0.01	+0.08
CC-Duelling-GNN	−0.92	−1.02	−0.93	−1.37	−2.03
LC-Simple	+0.52	+1.33	+1.55	+1.70	+6.26
LC-Duelling	−0.82	−1.26	−1.61	−1.53	−3.58
LC-Duelling-GNN	+0.00	+0.01	+0.00	−0.01	−0.13

The clamp changes the result significantly on three variants at every budget, and not in one direction. LC-Simple gains between 0.52 and 6.26 pp under the parity clamp, while CC-Duelling-GNN loses between 0.92 and 2.03 pp and LC-Duelling between 0.82 and 3.58 pp. These changes come from the attack arm rather than from the control. Ungated, the parity-clamp attack arm delivers 6.92 pp less than the full-clamp attack arm on LC-Simple, and 2.80 and 3.68 pp more on CC-Duelling-GNN and LC-Duelling, while the random control moves by less than one percentage point on all three. On CC-Duelling both arms move: ungated, the parity-clamp attack arm delivers 2.49 pp more, but its random control also delivers 1.82 pp more, and the net change of -0.67 pp is not significant. CC-Simple and CC-Simple-GNN change significantly only ungated, and by less than 0.25 pp, and LC-Duelling-GNN does not change.

In the coordinated scope the clamp again moves the results in both directions (Table 4.13). The parity-clamp adversary does more damage to CC-Duelling at every gated budget,

Table 4.13: Effect of the domain clamp in the coordinated scope: FGSM-parity minus full clamp adversarial gap, in percentage points, by victim variant and requested timing budget ($\varepsilon = 0.30$, $n = 30$ paired episodes). Positive values mean the parity-clamp adversary did more damage. Bold entries have a 95% confidence interval on the paired difference of Equation (3.18) excluding zero.

Variant	0.10	0.15	0.20	0.25	ungated
CC-Simple	+0.02	+0.06	+0.00	+0.03	+0.42
CC-Duelling	+0.46	+0.48	+0.68	+0.77	+0.46
CC-Simple-GNN	+0.01	+0.00	-0.01	+0.01	+0.11
CC-Duelling-GNN	+0.15	+0.33	+0.52	+0.27	-0.48
LC-Simple	-0.71	-0.57	-0.62	-0.77	-1.03
LC-Duelling	-0.34	-0.59	-0.75	-0.75	-2.67
LC-Duelling-GNN	+0.00	+0.01	+0.00	-0.01	-0.03

by 0.46 to 0.77 pp, and to CC-Simple when every step is attacked (+0.42 pp). It does less damage to LC-Duelling at every budget, by 0.34 to 2.67 pp, and to LC-Simple at three of the four gated budgets, by 0.62 to 0.77 pp. None of the three GNN variants shows a significant change.

CONCLUSIONS AND FUTURE WORK

This thesis asked whether the apparent robustness of MADDPG routing controllers to FGSM survives an observation-space attack that coordinates its perturbation across agents, inspired by SAJA, and divides it across time, inspired by PGD. The attack was realised as a learned adversary, trained against the frozen victim and evaluated against seven victim architectures in paired episodes with a random control. This chapter draws the conclusions of that evaluation on three questions, namely how effective the attack is compared with the FGSM baseline, what coordination adds over the independent attack, and what the timing budget achieves compared with attacking every step. It then sets out the limitations of those conclusions, above all their statistical relevance, and outlines the work that would strengthen or extend them.

5.1 Conclusions

5.1.1 Effectiveness compared with the FGSM baseline

The comparison with FGSM is only meaningful under the FGSM study’s own admissible set, and it was made on the 15 episodes that the two evaluations share (Section 4.3).

The learned attack did not do more total damage than FGSM. It exceeded the baseline significantly in a single configuration, the independent attack on every step against LC-Simple, by 2.21 pp. With the timing budget in place, the independent attack did significantly less damage than FGSM on four variants at every budget, which is partly by construction, since it acted on only 10 to 25% of the steps while FGSM acted on all of them. Coordination brought the attack closer to FGSM on the duelling variants: on LC-Duelling it could not be distinguished from FGSM at any budget, and on CC-Duelling from $b = 0.20$ onwards. When attacking every step, the coordinated attack did not differ significantly from FGSM on any variant. Where the learned attack has an advantage, it is per attacked step: on LC-Simple in the independent scope, and on CC-Duelling in the coordinated scope at $b = 0.25$, it caused more damage per step attacked than FGSM, without causing more damage in total.

The victim family withstands the learned adversary about as well as it withstands

the single-step baseline. The learned adversary is theoretically the stronger attacker: its policy class includes FGSM’s per-step rule, and it can additionally coordinate across agents and choose when to act. That it does no more damage than FGSM therefore supports the robustness of the controller, rather than suggesting that the FGSM baseline was too weak to reveal its vulnerabilities. This conclusion concerns the trained adversaries that were evaluated, not the best attack that is possible. A learned adversary is only as strong as its training, and one of them steered LC-Duelling to deliver more packets than it did unattacked (Section 4.2.1), which shows that a trained adversary can fail to find a damaging perturbation where one exists.

5.1.2 Coordination compared with the independent attack

Coordination changes the outcome of the attack, but it does not improve it uniformly. Under the FGSM study’s admissible set it changed the damage significantly on four victims, in both directions (Section 4.2.5). It increased the damage done to LC-Duelling at every budget, by 0.64 to 1.58 pp, turning a victim the independent adversary had helped into one the attack harms, and to CC-Duelling from $b = 0.15$ onwards, by up to 2.16 pp. It reduced the damage done to LC-Simple at every budget, by 0.48 to 3.99 pp, on the variant the independent attack had harmed most. Under the stricter full clamp the pattern was different again: coordination made LC-Simple attackable and reduced the damage done to CC-Duelling-GNN (Section 4.4.4).

The idea taken from SAJA, judging the perturbations of several agents together by their combined effect, is therefore able to reach damage that the independent attack does not, on some victims, while losing damage the independent attack had found on others. No single property of the victim explains which: the local-critic account that fits the full-clamp results does not fit the FGSM-parity ones. What coordination contributes depends on the pair of trained adversaries being compared as much as on the victim, and its gains do not come from changing more routing decisions.

5.1.3 Timing budget compared with attacking every step

The timing gate itself works as designed. It realised the requested budgets closely, at 10.0%, 15.0%, 19.9% and 24.9% of steps on average, and the trigger analysis of Sections 4.2.3 and 4.2.4 showed why the tie-rationing rule is needed. The critical-state score repeatedly takes the same value of 0.90, and without rationing the two tightest budgets would both have attacked between 17% and 21% of steps. In the FGSM-parity runs, where the reasons were recorded, the attacks fired only when the most utilised link was at least 75% utilised, and at the two tightest budgets almost entirely at 90% or more. The composition of the triggers was the same in both scopes, so it is set by the network and the budget rather than by the perturbation.

Spending a limited budget on those moments did not buy more total damage than attacking every step. For almost every victim the attack harmed, the ungated attack did the

most damage. What the budget can buy is efficiency: in three of the four settings evaluated, the damage per attacked step was higher under a tight budget than when attacking every step, by a factor of up to 2.6 on CC-Duelling-GNN under the full clamp and of about 1.3 on CC-Duelling in the coordinated scope under the FGSM study’s admissible set. It was not higher for the independent attack under that admissible set, where LC-Simple’s damage grew in proportion to the number of steps attacked. The PGD-inspired idea of dividing the attack across time therefore makes each attacked step count for more in most settings, but not reliably, and it does not make the attack more damaging overall. A limitation of the ungated comparison is that every adversary was trained with the gate at $b = 0.25$, so when attacking every step it acted on states it had not encountered during training.

5.1.4 Further observations

Three further results hold across the evaluation. The fraction of routing decisions an attack changes does not predict the damage it does: attacks that changed nearly the same fraction of decisions produced gaps several percentage points apart, so flip rates should not be reported as attack success. Which architectural choice appears vulnerable depends on the admissible set and the attack scope, so no single design can be named as the weak point of the controller. And two of the three GNN-encoded victims, CC-Simple-GNN and LC-Duelling-GNN, were not harmed by any attack in any setting.

5.2 Limitations

5.2.1 Statistical relevance of the results

The most important limitation concerns how far the results generalise beyond the particular adversaries that were trained. The learned adversary is not deterministic. Its actor and critic are initialised at random, exploration noise is added to its perturbations during training, and its updates are computed on randomly sampled minibatches, and training was not seeded, so two training runs with the same configuration produce different adversaries. Each of the 28 configurations of clamp, scope and victim was trained only once, and every result in Chapter 4 describes that single trained adversary.

The confidence intervals reported there are computed over 30 paired evaluation episodes, so they capture the variation between episodes for a given trained adversary, but not the variation between training runs. A significant gap therefore shows that the adversary that was trained beats its random control on these episodes, not that the attack configuration would do so reliably if it were trained again. The results most exposed to this are those that compare separately trained adversaries: the reversal on LC-Duelling, where the independent parity-clamp adversary improved delivery, the direction of the coordination effect, which compares the independent and coordinated adversaries, the effect of the domain clamp, which compares adversaries trained under each clamp, and the comparison with FGSM. The comparisons across timing budgets are less exposed,

since every budget, and the ungated run, evaluates the same trained adversary under a different gate.

Two more statistical caveats apply. All evaluations use a single traffic seed, and therefore the same 30 episodes, so the variation between traffic patterns is sampled only by those episodes. And the chapter reports well over a hundred confidence intervals, each at the 95% level and without correction for multiple comparisons, so a few cells can be expected to exclude zero by chance. Isolated significant cells, such as a single budget at which an otherwise null variant separates from zero, should be read with caution, and the conclusions of Section 5.1 rest on patterns that hold across budgets rather than on individual cells.

5.2.2 Scope of the evaluation

The evaluation covers a single topology, a single stressed traffic regime, one perturbation budget of $\varepsilon = 0.30$ with every agent compromised. The critical-state score was motivated partly by the thinning of redundancy after link failures. Each adversary was trained with the timing gate at $b = 0.25$ only, so the tighter budgets act on a subset of its training states, and the ungated evaluation asks it to act on states it never encountered. The reasons why the timing gate fired were recorded only for the FGSM-parity runs.

5.2.3 Design of the attack and its control

The adversary is rewarded with the fraction of forwarding events that end in a drop, a per-hop proxy for the end-to-end delivery on which it is scored, and the two can come apart. Inspection of the trained adversaries also indicates that their outputs sit close to the bounds of $[-1, 1]$ for most features, so the attack perturbs most features by nearly the full ε , whereas the random control draws each feature's perturbation uniformly from $[-\varepsilon, \varepsilon]$. The adversarial gap therefore reflects the size of the perturbation as well as its direction, and a random control that also perturbs every feature by $\pm\varepsilon$, as the FGSM baseline's did, would isolate direction more cleanly. Finally, the threat model assumes that the attacker can train against a faithful simulator of the network and, when the attack is timed, read the utilisation of every link while it attacks.

5.3 Future Work

5.3.1 Repeated training for statistical relevance

The limitation of Section 5.2.1 can be addressed directly. The evaluation should be repeated with several independently trained adversaries per configuration, each trained from a different random seed, so that the spread between training runs can be measured and a result can be attributed to the attack rather than to one run.

5.3.2 Optimisation-based PGD and SAJA attacks

This thesis took PGD and SAJA as inspiration, but realised both mechanisms through a learned adversary rather than by solving an optimisation problem at each step. A natural next step is to implement the attacks in their original form, as gradient optimisation problems, and to evaluate them under the same protocol. A PGD attack [14] would solve for each attacked step's perturbation by iterated projected gradient ascent on an attack objective, and could be combined with the same timing gate. A SAJA-style attack [18] would perturb all compromised agents' observations jointly by gradient ascent, scoring each candidate by the centralised critic's value of the joint action it induces.

Comparing these attacks with the learned adversary and with FGSM, within the same admissible set and on the same paired episodes, would show whether learning the adversary is beneficial for adversarial attacks on this controller, or whether direct optimisation achieves more damage. Such a study would have to address several obstacles. The routing decision is an $\arg \max$ over candidate paths, so its gradient vanishes almost everywhere and must be approximated, for instance with a straight-through estimator. The local-critic variants provide no joint critic with which to score a coordinated perturbation, so a different joint objective would be needed for them. And the GNN encoder couples the agents' observations, so the gradient for one agent depends on the others'. It would also have to account for the difference in cost, since an optimisation-based attack requires many gradient evaluations of the victim at every attacked step, whereas the learned adversary needs only a single forward pass once trained and never needs the victim's gradients.

BIBLIOGRAPHY

- [1] The L^AT_EX Project team. *L^AT_EX - A document preparation system*. 1985. URL: <https://www.latex-project.org> (visited on 2025-12-29) (cit. on p. iii).
- [2] J. M. Lourenço. *The aidisclose package: Generative AI disclosure checklist and statements*. Version 1.6.2. 2025. URL: <https://ctan.org/pkg/aidisclose> (cit. on p. iii).
- [3] Y. Suchikova, N. Tsybuliak, J. A. T. da Silva, and S. Nazarovets. “GAIDeT (Generative AI Delegation Taxonomy): A Taxonomy for Humans to Delegate Tasks to Generative Artificial Intelligence in Scientific Research and Publishing”. In: *Accountability in Research* (2025), pp. 1–27. DOI: 10.1080/08989621.2025.2544331 (cit. on p. iii).
- [4] C. Tian, H. Cao, J. Xie, S. Garg, J. Rodrigues, and M. S. Hossain. “MADDPG-empowered slice reconfiguration approach for 5G multi-tier system”. In: *Journal of Network and Computer Applications* 219 (2023-09), p. 103725. DOI: 10.1016/j.jnca.2023.103725 (cit. on pp. 1, 7).
- [5] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch. “Multi-agent actor-critic for mixed cooperative-competitive environments”. In: *Advances in neural information processing systems* 30 (2017) (cit. on pp. 1, 7, 19, 21).
- [6] P. Almasan, J. Suárez-Varela, K. Rusek, P. Barlet-Ros, and A. Cabellos-Aparicio. “Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case”. In: *Computer Communications* 196 (2022-12), pp. 184–194. ISSN: 0140-3664. DOI: 10.1016/j.comcom.2022.09.029 (cit. on p. 1).
- [7] H. Wang, Y. Liu, W. Li, and Z. Yang. “Multi-agent deep reinforcement learning-based fine-grained traffic scheduling in data center networks”. In: *Future Internet* 16.4 (2024), p. 119 (cit. on p. 1).
- [8] W. Huang, H. Liu, Y. Li, and L. Ma. “ERA-MADDPG: An Elastic Routing Algorithm Based on Multi-Agent Deep Deterministic Policy Gradient in SDN”. In: *Future Internet* 17.7 (2025), p. 291 (cit. on pp. 1, 7).

- [9] S. S. Bhavanasi, L. Pappone, and F. Esposito. "Routing with Graph Convolutional Networks and Multi-Agent Deep Reinforcement Learning". In: *2022 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. 2022, pp. 72–77. DOI: 10.1109/NFV-SDN56302.2022.9974607 (cit. on p. 1).
- [10] W. Zhang, C. Liu, Y. Pi, Y. Zhang, H. Huang, B. Rao, Y. Ding, S. Yang, and J. Jiang. *DRAMA: A Dynamic Packet Routing Algorithm using Multi-Agent Reinforcement Learning with Emergent Communication*. 2025. arXiv: 2504.04438 [cs.NI] (cit. on pp. 1, 7).
- [11] F. Alanazi and M. Zareei. "Multi-Agent Deep Reinforcement Learning for Dynamic Routing in MANETs Using Graph Neural Networks". In: *IEEE Access* 13 (2025), pp. 152469–152478. DOI: 10.1109/ACCESS.2025.3601994 (cit. on p. 1).
- [12] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. *Intriguing properties of neural networks*. 2014. arXiv: 1312.6199 [cs.CV] (cit. on pp. 1, 9).
- [13] I. J. Goodfellow, J. Shlens, and C. Szegedy. *Explaining and Harnessing Adversarial Examples*. 2015. arXiv: 1412.6572 [stat.ML] (cit. on pp. 1, 10, 20).
- [14] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks*. 2019. arXiv: 1706.06083 [stat.ML] (cit. on pp. 1, 2, 10, 23, 27, 63).
- [15] N. Carlini and D. Wagner. *Towards Evaluating the Robustness of Neural Networks*. 2017. arXiv: 1608.04644 [cs.CR] (cit. on pp. 1, 12).
- [16] H. Zhang, H. Chen, C. Xiao, B. Li, M. Liu, D. Boning, and C.-J. Hsieh. "Robust Deep Reinforcement Learning against Adversarial Perturbations on State Observations". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 21024–21037 (cit. on pp. 1, 3, 11, 20, 23, 29).
- [17] P. Amaral and D. Raimundo. "Multi-Agent Deep Reinforcement Learning for Adaptive Routing: A Study of Critic Design and Graph Neural Network State Encoding". In: (2021) (cit. on pp. 2, 17, 19, 21).
- [18] W. Guo, G. Liu, and Z. Zhou. *SAJA: A State-Action Joint Attack Framework on Multi-Agent Deep Reinforcement Learning*. 2025. arXiv: 2510.13262 [cs.AI] (cit. on pp. 2, 10, 12, 13, 23, 25, 63).
- [19] T. Theodoropoulos, D. Kafetzis, J. Violos, A. Makris, and K. Tserpes. "Multi-agent deep reinforcement learning for weighted multi-path routing". In: *Proceedings of the 3rd Workshop on Flexible Resource and Application Management on the Edge*. 2023, pp. 7–11 (cit. on p. 5).
- [20] M. M. Alam and S. Moh. "Joint Trajectory Control, Frequency Allocation, and Routing for UAV Swarm Networks: A Multi-Agent Deep Reinforcement Learning Approach". In: *IEEE Transactions on Mobile Computing* 23.12 (2024), pp. 11989–12005. DOI: 10.1109/TMC.2024.3403890 (cit. on p. 5).

- [21] F. Lozano-Cuadra, M. D. Thorsager, I. Leyva-Mayorga, and B. Soret. *An open source Multi-Agent Deep Reinforcement Learning Routing Simulator for satellite networks*. 2024. arXiv: 2407.11047 [cs.LG] (cit. on p. 5).
- [22] P. Liu, X. Bai, and H. Cheng. “DRL-MR: Multipath Routing Based on Multi-Agent Deep Reinforcement Learning for SDN-Based Data Center Networks”. In: *2025 International Joint Conference on Neural Networks (IJCNN)*. 2025, pp. 1–7. doi: 10.1109/IJCNN64981.2025.11227800 (cit. on p. 5).
- [23] R. A. Alliche, R. A. Pardo, and L. Sassatelli. “O-DQR: A Multi-Agent Deep Reinforcement Learning for Multihop Routing in Overlay Networks”. In: *IEEE Transactions on Network and Service Management* 22.1 (2025), pp. 439–455. doi: 10.1109/TNSM.2024.3485196 (cit. on p. 5).
- [24] Z. Jia, S. He, Q. Zhu, W. Wang, Q. Wu, and Z. Han. “Trusted Routing for Blockchain-Empowered UAV Networks via Multi-Agent Deep Reinforcement Learning”. In: *IEEE Transactions on Communications* 73.12 (2025), pp. 14227–14242. doi: 10.1109/TCOMM.2025.3597655 (cit. on p. 5).
- [25] Z. Zhou, G. Liu, W. Guo, and M. Zhou. “Adversarial Attacks on Multiagent Deep Reinforcement Learning Models in Continuous Action Space”. In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 54.12 (2024), pp. 7633–7646. doi: 10.1109/TSMC.2024.3454118 (cit. on pp. 5, 7).
- [26] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG] (cit. on p. 6).
- [27] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. “Human-level control through deep reinforcement learning”. In: *nature* 518.7540 (2015), pp. 529–533 (cit. on p. 6).
- [28] X. Geng and B. Zhang. “Deep Q-Network-Based Intelligent Routing Protocol for Underwater Acoustic Sensor Network”. In: *IEEE Sensors Journal* 23.4 (2023), pp. 3936–3943. doi: 10.1109/JSEN.2023.3234112 (cit. on p. 6).
- [29] Q. Fu, E. Sun, K. Meng, M. Li, and Y. Zhang. “Deep Q-Learning for Routing Schemes in SDN-Based Data Center Networks”. In: *IEEE Access* 8 (2020), pp. 103491–103499. doi: 10.1109/ACCESS.2020.2995511 (cit. on p. 6).
- [30] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas. *Dueling Network Architectures for Deep Reinforcement Learning*. 2016. arXiv: 1511.06581 [cs.LG] (cit. on pp. 6, 22).
- [31] M. Zhao. *Breaking the Cold-Start Barrier: Reinforcement Learning with Double and Dueling DQNs*. 2025. arXiv: 2508.21259 [cs.IR] (cit. on p. 6).

- [32] J. Zhang, F. Qian, and J. Yang. "Online routing and spectrum allocation in elastic optical networks based on dueling Deep Q-network". In: *Computers & Industrial Engineering* 173 (2022), p. 108663. ISSN: 0360-8352. DOI: <https://doi.org/10.1016/j.cie.2022.108663> (cit. on p. 6).
- [33] N. H. Pham, L. M. Nguyen, J. Chen, H. T. Lam, S. Das, and L. Weng. "c-MBA: Adversarial attack for cooperative MARL using learned dynamics model". In: *NeurIPS ML Safety Workshop*. 2022 (cit. on p. 7).
- [34] S. Li, J. Guo, J. Xiu, Y. Zheng, P. Feng, X. Yu, J. Wang, A. Liu, Y. Yang, B. An, et al. "Attacking cooperative multi-agent reinforcement learning by adversarial minority influence". In: *Neural Networks* (2025), p. 107747 (cit. on pp. 7, 13).
- [35] S. Kuang, J. Zheng, S. Liang, Y. Li, S. Liang, and W. Huang. "RS-MADDPG: Routing Strategy Based on Multi-Agent Deep Deterministic Policy Gradient for Differentiated QoS Services". In: *Future Internet* 17.9 (2025), p. 393 (cit. on p. 7).
- [36] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun. *Graph Neural Networks: A Review of Methods and Applications*. 2021. arXiv: 1812.08434 [cs.LG] (cit. on p. 8).
- [37] Z. Zhang, P. Cui, and W. Zhu. "Deep Learning on Graphs: A Survey". In: *IEEE Transactions on Knowledge and Data Engineering* 34.1 (2022), pp. 249–270. DOI: 10.1109/TKDE.2020.2981333 (cit. on p. 8).
- [38] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu. "A Comprehensive Survey on Graph Neural Networks". In: *IEEE Transactions on Neural Networks and Learning Systems* 32.1 (2021), pp. 4–24. DOI: 10.1109/TNNLS.2020.2978386 (cit. on p. 8).
- [39] I. Chami, S. Abu-El-Haija, B. Perozzi, C. Ré, and K. Murphy. "Machine learning on graphs: A model and comprehensive taxonomy". In: *Journal of Machine Learning Research* 23.89 (2022), pp. 1–64 (cit. on p. 8).
- [40] W. Jiang. "Graph-based deep learning for communication networks: A survey". In: *Computer Communications* 185 (2021-03), pp. 40–54. ISSN: 0140-3664. DOI: 10.1016/j.comcom.2021.12.015 (cit. on p. 8).
- [41] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio. "Unveiling the potential of Graph Neural Networks for network modeling and optimization in SDN". In: *Proceedings of the 2019 ACM Symposium on SDN Research*. SOSR '19. San Jose, CA, USA: Association for Computing Machinery, 2019, pp. 140–151. ISBN: 9781450367103. DOI: 10.1145/3314148.3314357 (cit. on p. 8).
- [42] H. Wei, Y. Zhao, and K. Xu. "G-Routing: graph neural networks-based flexible online routing". In: *IEEE Network* 37.4 (2023), pp. 90–96 (cit. on p. 8).

-
- [43] M. Ding, Y. Guo, Z. Huang, B. Lin, and H. Luo. "GROM: A generalized routing optimization method with graph neural network and deep reinforcement learning". In: *J. Netw. Comput. Appl.* 229.C (2024-09). ISSN: 1084-8045. DOI: 10.1016/j.jnca.2024.103927 (cit. on p. 8).
 - [44] F. Zhou, X. Liu, R. Yu, and G. Xue. *Traffic Engineering in Large-scale Networks with Generalizable Graph Neural Networks*. 2025. arXiv: 2503.24203 [cs.NI] (cit. on p. 8).
 - [45] W. Jiang, H. Han, Y. Zhang, J. Wang, M. He, W. Gu, J. Mu, and X. Cheng. "Graph neural networks for routing optimization: Challenges and opportunities". In: *Sustainability* 16.21 (2024), p. 9239 (cit. on p. 8).
 - [46] A. Alzubaidi, A. S. Al-Sumaiti, and M. Khonji. "Adversarial Deep Reinforcement Learning Attacks on Multi-Agent Autonomous Cooperative Driving Policies". In: *IET Intelligent Transport Systems* 19.1 (2025), e70066. DOI: <https://doi.org/10.1049/itr2.70066> (cit. on p. 9).
 - [47] Z. Zhou, G. Liu, W. Guo, and M. Zhou. "Adversarial Attacks on Multiagent Deep Reinforcement Learning Models in Continuous Action Space". In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 54.12 (2024), pp. 7633–7646. DOI: 10.1109/TSMC.2024.3454118 (cit. on p. 9).
 - [48] S. G. Finlayson, J. D. Bowers, J. Ito, J. L. Zittrain, A. L. Beam, and I. S. Kohane. "Adversarial attacks on medical machine learning". In: *Science* 363.6433 (2019), pp. 1287–1289. DOI: 10.1126/science.aaw4399 (cit. on p. 9).
 - [49] W. Villegas-Ch, A. Jaramillo-Alcázar, and S. Luján-Mora. "Evaluating the robustness of deep learning models against adversarial attacks: An analysis with fgsm, pgd and cw". In: *Big Data and Cognitive Computing* 8.1 (2024), p. 8 (cit. on p. 10).
 - [50] N. Akhtar, A. Mian, N. Kardan, and M. Shah. "Advances in adversarial attacks and defenses in computer vision: A survey". In: *IEEE access* 9 (2021), pp. 155161–155196 (cit. on p. 10).
 - [51] A. Kurakin, I. Goodfellow, and S. Bengio. *Adversarial examples in the physical world*. 2016. arXiv: 1607.02533 [cs.CV] (cit. on p. 10).
 - [52] A. Rozsa, E. M. Rudd, and T. E. Boulton. "Adversarial diversity and hard positive generation". In: *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*. 2016, pp. 25–32 (cit. on p. 10).
 - [53] C. Xie, Z. Zhang, Y. Zhou, S. Bai, J. Wang, Z. Ren, and A. L. Yuille. "Improving Transferability of Adversarial Examples With Input Diversity". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019-06 (cit. on p. 10).
 - [54] A. Kurakin, I. Goodfellow, and S. Bengio. *Adversarial Machine Learning at Scale*. 2017. arXiv: 1611.01236 [cs.CV] (cit. on p. 10).

- [55] H. Waghela, J. Sen, and S. Rakshit. “Robust Image Classification: Defensive Strategies against FGSM and PGD Adversarial Attacks”. In: *2024 Asian Conference on Intelligent Technologies (ACOIT)*. IEEE, 2024-09, pp. 1–7. doi: 10.1109/acoit62457.2024.10941671 (cit. on p. 10).
- [56] H. Zhang, H. Chen, D. Boning, and C.-J. Hsieh. “Robust Reinforcement Learning on State Observations with Learned Optimal Adversary”. In: *International Conference on Learning Representations*. 2021 (cit. on p. 11).
- [57] Y. Sun, R. Zheng, Y. Liang, and F. Huang. “Who is the strongest enemy? towards optimal and efficient evasion attacks in deep rl”. In: *arXiv preprint arXiv:2106.05087* (2021) (cit. on p. 12).
- [58] Y.-C. Lin, Z.-W. Hong, Y.-H. Liao, M.-L. Shih, M.-Y. Liu, and M. Sun. “Tactics of Adversarial Attack on Deep Reinforcement Learning Agents”. In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-17)*. 2017. arXiv: 1703.06748 [cs.LG] (cit. on pp. 12, 27).
- [59] S. Lee, J. Hwang, Y. Jo, and S. Han. *Wolfpack Adversarial Attack for Robust Multi-Agent Reinforcement Learning*. 2025. arXiv: 2502.02844 [cs.LG] (cit. on p. 13).
- [60] Z. Wu, S. C. Ye, B. Han, and M. Gombolay. “Hijacking Robot Teams Through Adversarial Communication”. In: *7th Annual Conference on Robot Learning*. 2023 (cit. on p. 13).
- [61] C. Zhao, Y. Ze, J. Dong, B. Wang, and S. Li. “DPMAC: Differentially private communication for cooperative multi-agent reinforcement learning”. In: *arXiv preprint arXiv:2308.09902* (2023) (cit. on p. 13).
- [62] L. Yuan, F. Chen, Z. Zhang, and Y. Yu. *Communication-Robust Multi-Agent Learning by Adaptable Auxiliary Multi-Agent Adversary Generation*. 2023. arXiv: 2305.05116 [cs.LG] (cit. on p. 13).
- [63] W. Shen, H. Li, and Z. Zheng. “Coordinated attacks against federated learning: A multi-agent reinforcement learning approach”. In: *ICLR 2021 Workshop on Security and Safety in Machine Learning Systems (SecML)*. ICLR 2021 Workshop on Security and Safety in Machine Learning Systems (SecML). 2021 (cit. on p. 13).
- [64] J. Dai and H. Sun. *Effective backdoor attack on graph neural networks in link prediction tasks*. 2025. arXiv: 2401.02663 [cs.LG] (cit. on p. 13).
- [65] W. Guo, G. Liu, Z. Zhou, and L. Wang. “PNAct: Crafting Backdoor Attacks in Safe Reinforcement Learning”. In: *arXiv preprint arXiv:2507.00485* (2025) (cit. on p. 13).
- [66] J. Guo, A. Li, L. Wang, and C. Liu. “Policycleanse: Backdoor detection and mitigation for competitive reinforcement learning”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 4699–4708 (cit. on p. 13).
- [67] S. Chen, Y. Qiu, and J. Zhang. *Backdoor Attacks on Multiagent Collaborative Systems*. 2022. arXiv: 2211.11455 [cs.MA] (cit. on p. 13).

- [68] X. Pan, Y. Cao, X. Wu, E. Zelikman, C. Xiao, Y. Sui, R. Chakraborty, and R. S. Fearing. “Transferable adversarial attacks on deep reinforcement learning with domain randomization”. In: (2020) (cit. on p. 14).
- [69] L. Luo, W. Ren, H. Huang, and F. Wang. “A Survey on Privacy Attacks and Defenses in Graph Neural Networks”. In: *Information Technology and Control* 53.4 (2024), pp. 1251–1278 (cit. on p. 15).
- [70] J. Wei, L. Yaxin, X. Han, W. Yiqi, J. Shuiwang, and A. Charu. “Adversarial attacks and defenses on graphs: A review, a tool, and empirical studies”. In: *arXiv preprint arXiv:2003.00653* 653 (2020) (cit. on p. 15).
- [71] D. Zügner, A. Akbarnejad, and S. Günnemann. “Adversarial Attacks on Neural Networks for Graph Data”. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD ’18. London, United Kingdom: Association for Computing Machinery, 2018, pp. 2847–2856. ISBN: 9781450355520. doi: 10.1145/3219819.3220078 (cit. on p. 15).
- [72] M. Aburidi and R. Marcia. “Topological adversarial attacks on graph neural networks via projected meta learning”. In: (2024) (cit. on p. 15).

TIMING GATE TRIGGERS

This appendix gives the heatmaps of why the timing gate fired in the FGSM-parity evaluations. They are discussed in Sections 4.2.3 and 4.2.4.

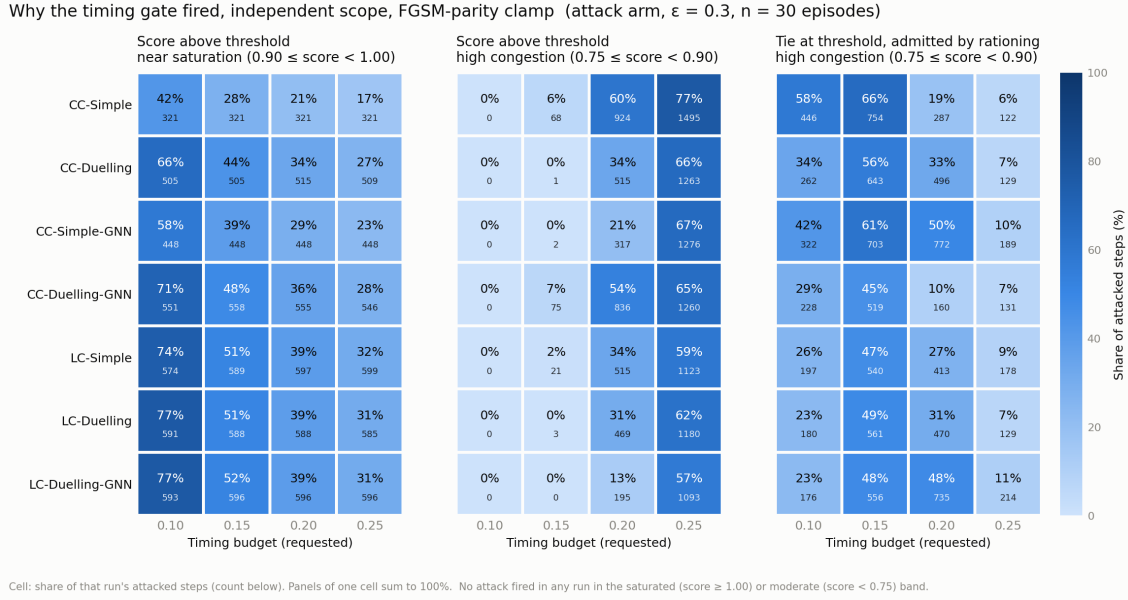


Figure A.1: Independent scope, FGSM-parity domain clamp: why the timing gate fired in the attack arm ($\epsilon = 0.30$, $n = 30$ episodes). One panel per trigger that occurred in at least one run, combining the case of Equation (3.10) that admitted the step with the congestion band of its score. Rows are victim variants and columns the requested timing budget. Each cell gives the trigger's share of that run's attacked steps, with the count beneath it, so the three panels of one cell sum to 100%. Ungated runs have no gate and are omitted.

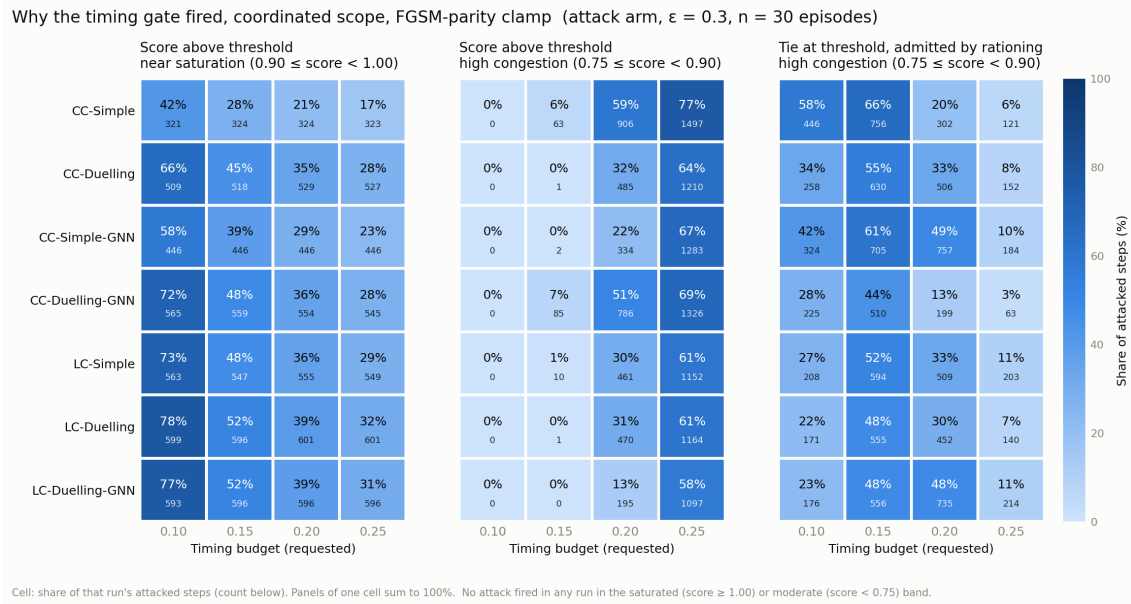


Figure A.2: Coordinated scope, FGSM-parity domain clamp: why the timing gate fired in the attack arm ($\epsilon = 0.30$, $n = 30$ episodes). Panels, rows, columns and cell contents as in Figure A.1.



2026

A Very Long and Impressive Thesis Title With a Forced Line Break Miguel Chen